Convolutional Neural Networks

With some materials from Fuxin Li, Fei-Fei Li

Rasha Obeidat, PhD

AI342

Jordan University of Science and Technology

Introduction

- **Computer Vision**: the interdisciplinary field that aims to develop techniques that help computers understand and interpret the visual data such as digital images or videos.
- There is massive amounts of visual data are being produced daily due to the increasing use of sensors.
- **300 hours** of video are uploaded to YouTube **every minute**!

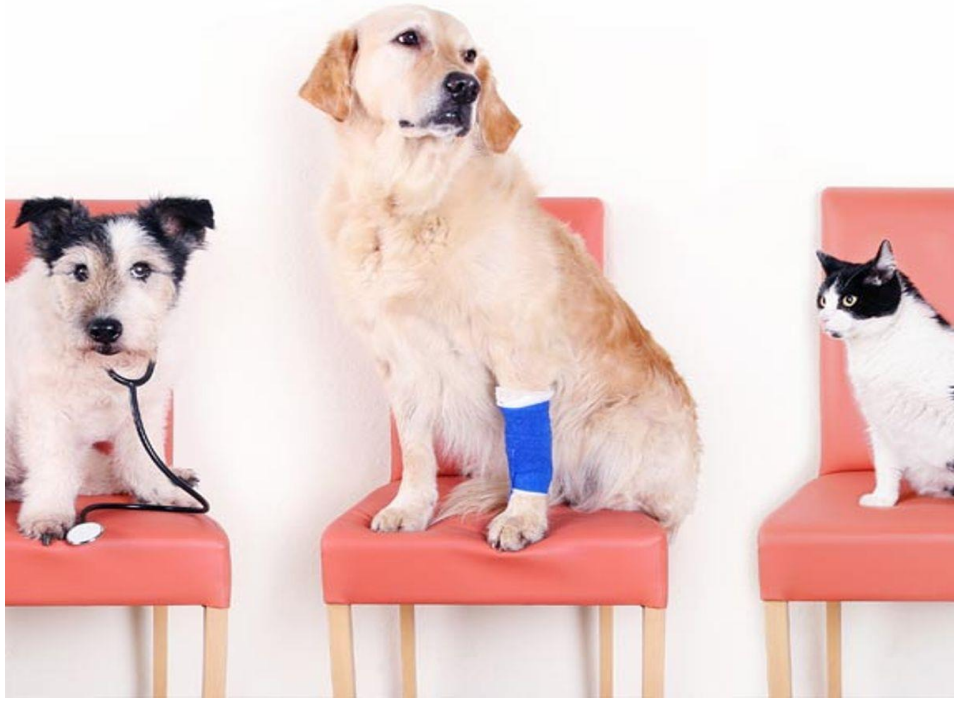
  There should be an automatic way to understand and annotate these videos.

Visual recognition problems:

- Image classification
- Object detection
- Image captioning
- Visual question answering
- Action classification
- Object tracking
- Activity recognition
- Image retrieval
- Etc.

Image classification

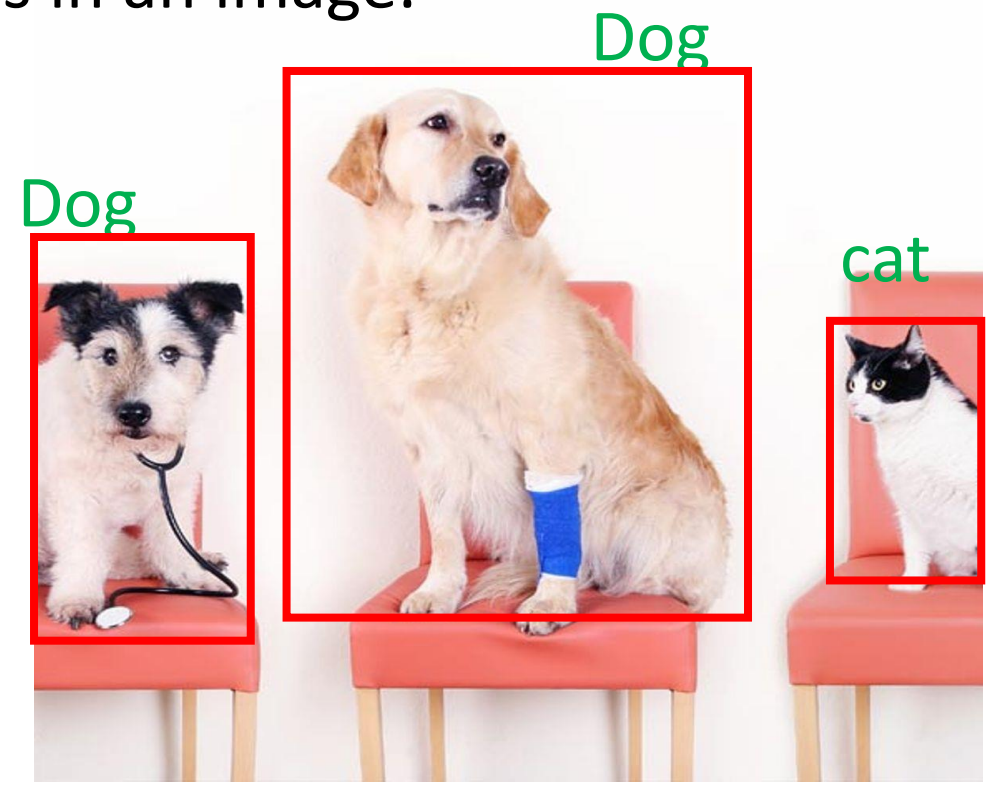
- Tagging an image with one or more classes from a predefined set of classes.



Classes: Dog , Cat

Object Detection

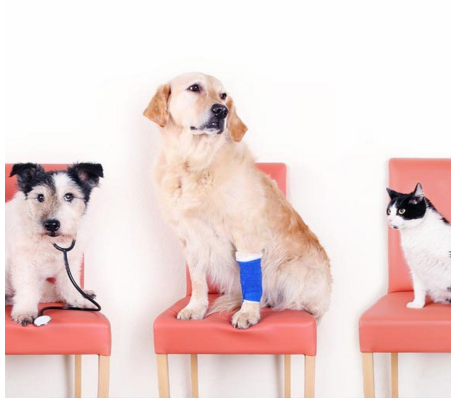
- **Detecting** instances of **objects** (by drawing a box around it) from a particular class in an image.



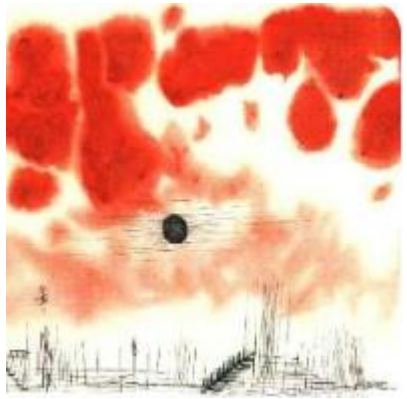
Neural Style Transfer

- Apply the artistic style of one image to the content of another image

Content image



Style image



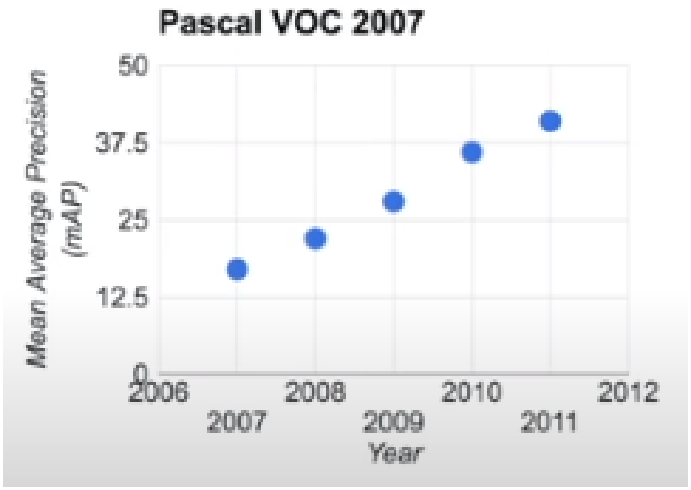
The PASCAL Visual Object Classes(VOC) Challenge

- Data evolved with time from using 4 classes (2005) to 20 classes (2012).
- Classification and Detection competitions.
- Enabled researcher to measure the progress in Object Recognition.

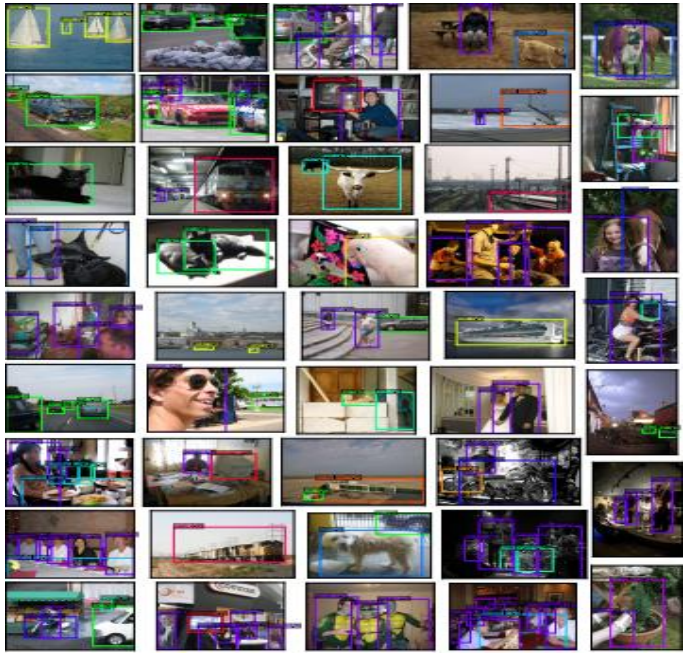
PASCAL2
Pattern Analysis, Statistical Modelling and Computational Learning

2005-2012

VOC2005, Khashif et al. (2017)



Fie-Fie Li: History of CV



VOC2012 Liu et al. (2017)

Recognizing Handwritten Digits

MNIST dataset

60,000 training, 10,000 testing

Algorithm	Error Rate (%)
Linear classifier (perceptron)	12.0
K-nearest-neighbors	5.0
Boosting	1.26
SVM	1.4
Neural Network	1.6
Convolutional Neural Networks	0.95
With automatic distortions + ensemble + many tricks	0.23

38

Source:http://classes.engr.oregonstate.edu/eecs/winter2018/cs535/Slides/1B_Introduction_2018.pdf

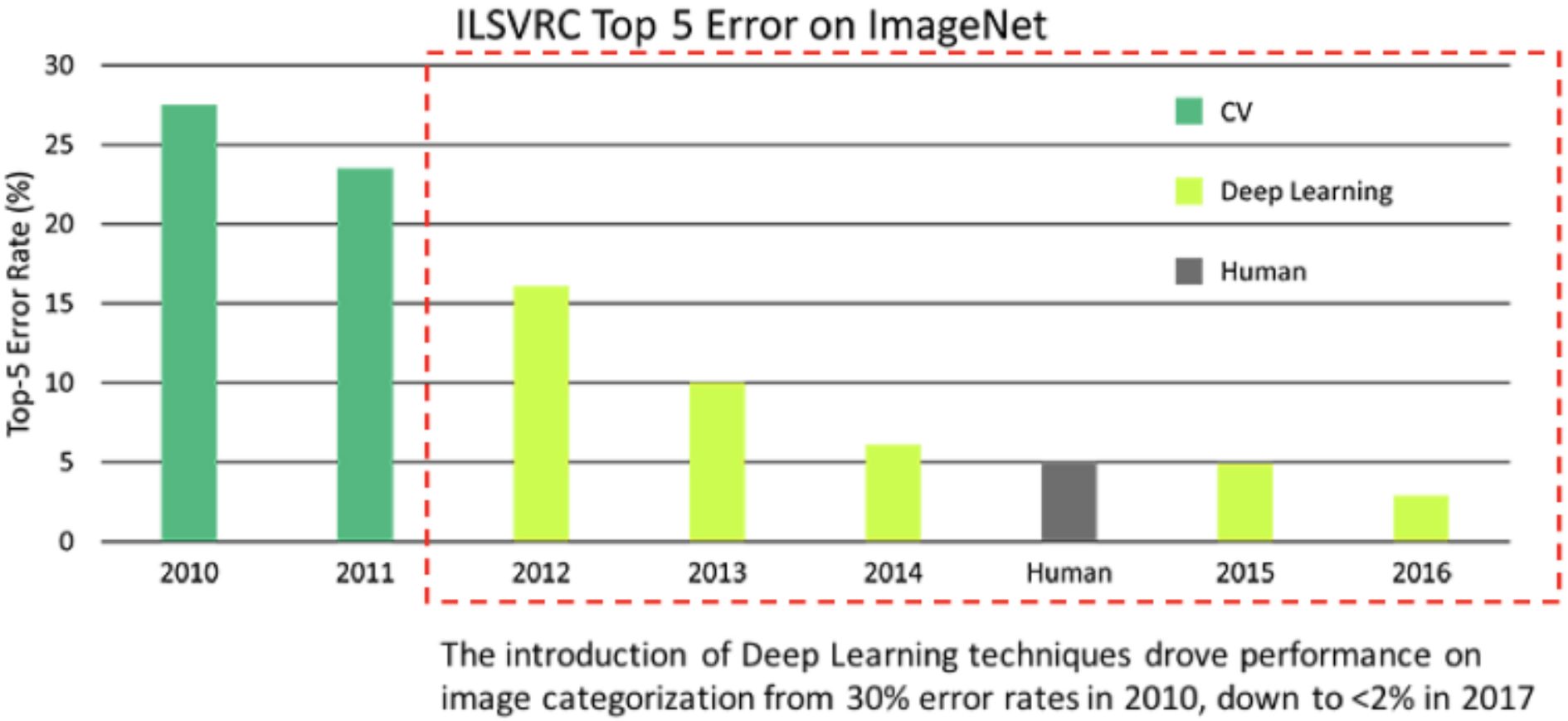
IMAGENET

- Large visual dataset designed for use in visual Object Recognition research.
- Hand-annotated 14 million images under 22,000 class.
- One million images with bounding boxes.
- Has been built starting from of WordNet.

The annual competition is called **ILSVRC**: ImageNet Large Scale Visual Recognition Challenge

IMAGENET

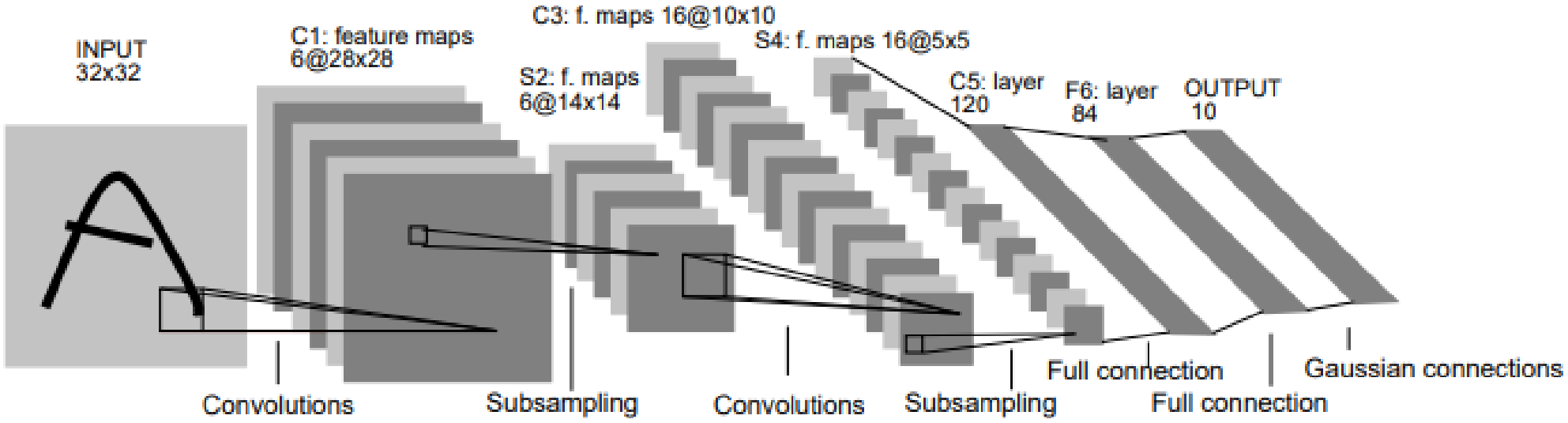
ILSVRC



Source: https://www.freecodecamp.org/news/everything-you-need-to-know-to-master-convolutional-neural-networks-ef98ca3c7655/

CNN History

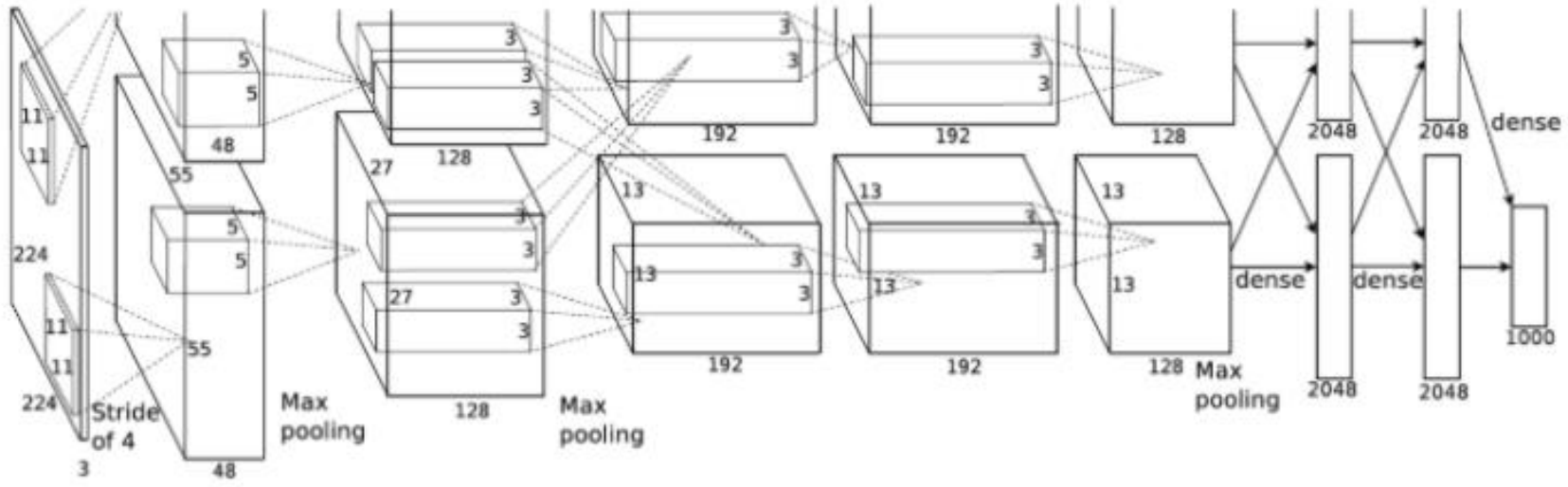
- The invention of CCN dates **back to the 1988 by** Yann LeCune and his collaborators at Bell labs.
- **LeNet5**: LeCun et al. (1988)

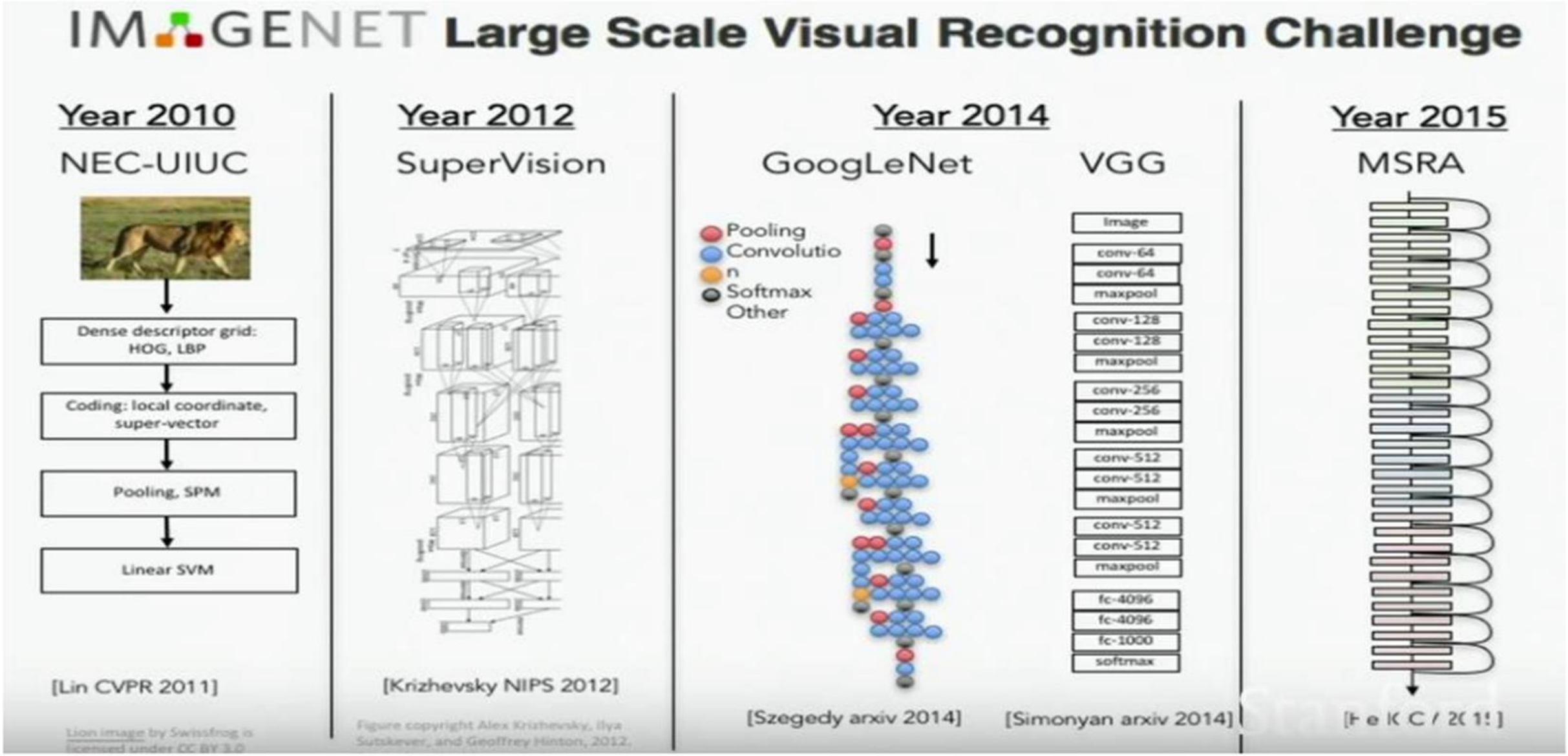


- **Challenges in the 1990s**: Despite the success of LeNet5, neural networks faced significant challenges:
  - **Limited computing power** and **lack of large datasets** hindered the practical application of CNNs.
  - **Training deep networks** was still difficult due to issues like vanishing gradients and limited available computational resources.

CNN History

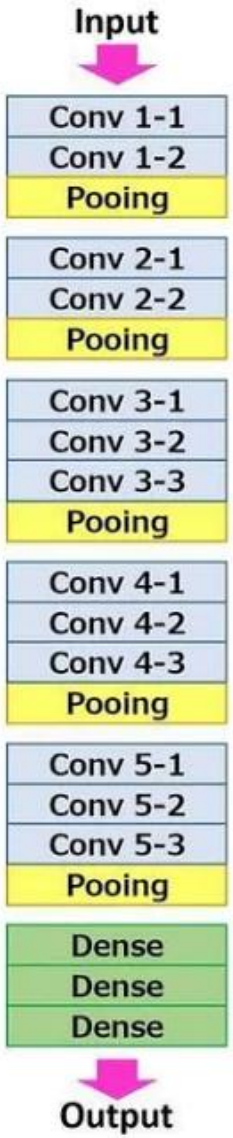
- In 2010 Ciresan et al. released the implementations of GPU Neural nets.
- In 2012, AlexNet (by Alex Krizhevsky) won the ImageNet competition by a large performance margin.



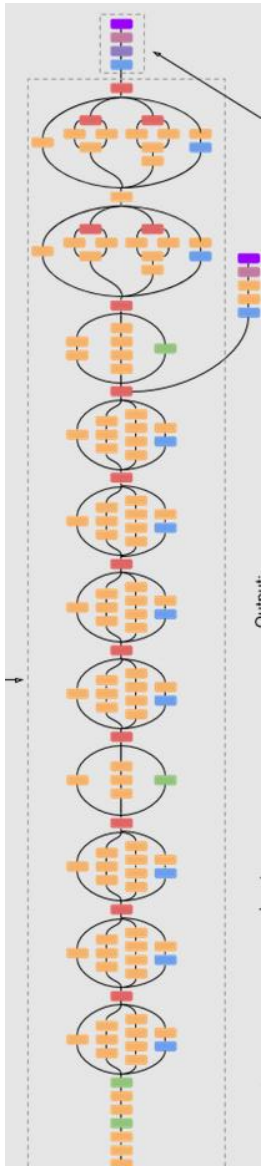


Source: https://dbs.uni-leipzig.de/file/Stur_Ausarbeitung.pdf

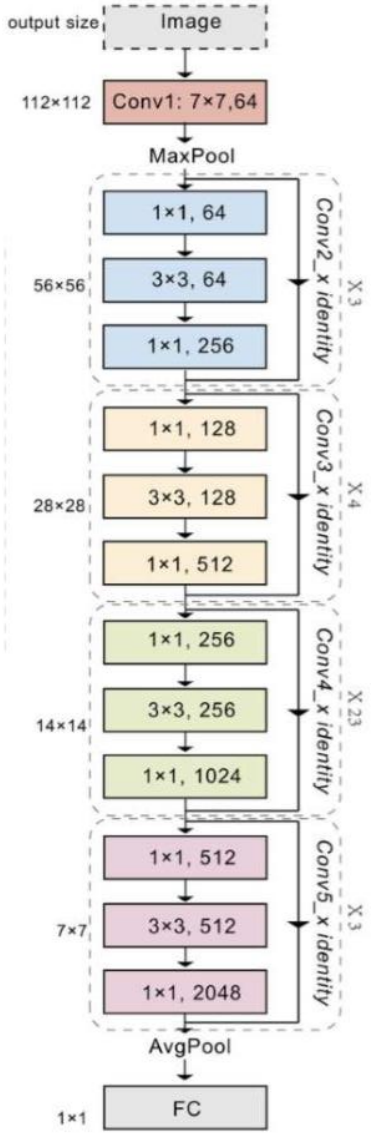
VGG (2014)



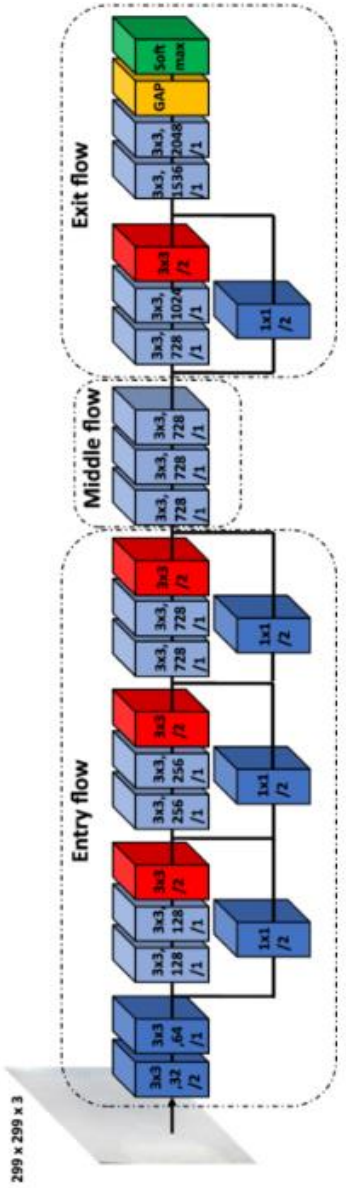
Inception V3 (2015)



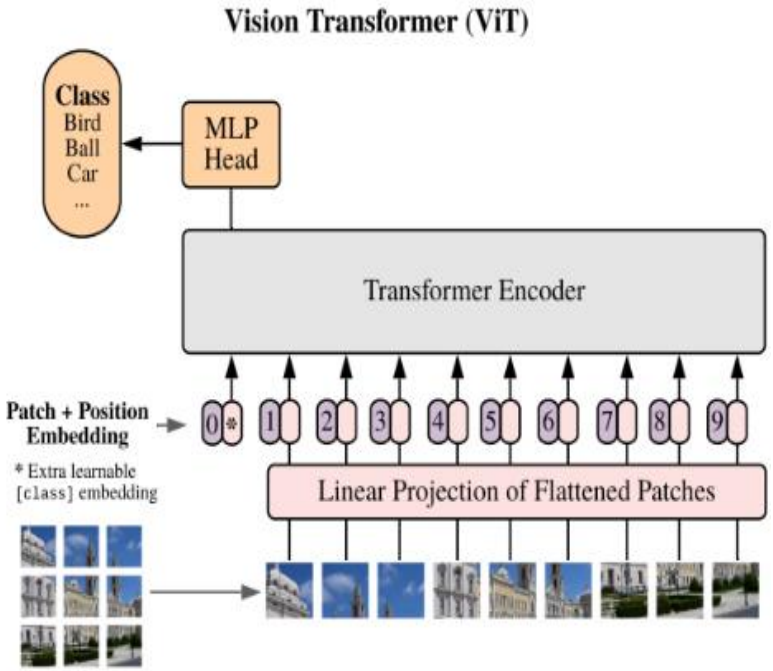
ResNet (2015)



Xception (2017)



Transformer-based Vision (ViT, 2020)



CNNs gain popularity after 2010, Why?

- Computational Power: faster computers, GPUs.
- More data becomes available.
- Improvements in optimization and regularizations:
  - ReLU
  - Maxpooling
  - Dropout (Srivastava 2014).
  - Etc.

Convolution Operation

Motivation: Sobel filters to detect edges

Sobel filters to detect edges

Sobel filter measures the varying pixel intensity in an image.

- **Horizontal Sobel Filter** $(\boldsymbol{G_x})$**:** detects vertical edges by emphasizing vertical changes in pixel intensity, which correspond to **vertical edges**.
- **Vertical Sobel Filter** $(\boldsymbol{G_y})$**:** detects horizontal edges by emphasizing horizontal changes in pixel intensity which correspond to horizontal edges.

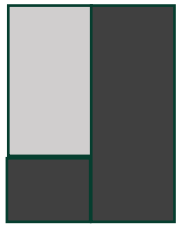
$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Sobel filters to detect edges

- Key features of this Filter ($\boldsymbol{G_x}$) :
  - **Left column**: Contains negative values $(-1, -2, -1)$.
  - **Middle column**: Contains zeros, meaning no contribution from the central pixels.
  - **Right column**: Contains positive values $(1, 2, 1)$.
- **Mechanism of Edge Detection:w**hen the filter slides over an image:
  - For regions with constant pixel intensity , the sum of pixel differences across the filter will be close to zero. This is because the positive and negative contributions will cancel each other out.
- **Areas with sharp horizontal intensity changes (vertical edge)**:
  - In regions where there's a transition from light to dark (or vice versa) along the horizontal axis:
    - The left side of the kernel (negative values) will overlap with higher−intensity pixels (or lower).
    - The right side of the kernel (positive values) will overlap with lower−intensity pixels (or higher).
    - This results in a large negative (or positive)magnitude output, indicating the presence of an edge.

Original image



9	9	9	0	0	0
9	9	9	0	0	0
9	9	9	0	0	0
9	9	9	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Conv. Operation

Sobel filters

*

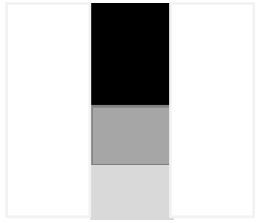
-1	0	1
-2	0	2
-1	0	1

=

Activation map for edges.

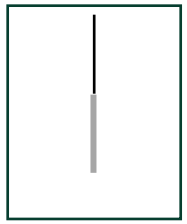
0	-36	-36	0
0	-36	-36	0
0	-27	-27	0
0	-9	-9	0

Resultant image



$$\begin{bmatrix} 255 & 0 & 0 & 255 \\ 255 & 0 & 0 & 255 \\ 255 & 64 & 64 & 255 \\ 255 & 191 & 191 & 255 \end{bmatrix}$$

In a larger image (padding)



Scale the values to the range **[0, 255]**:

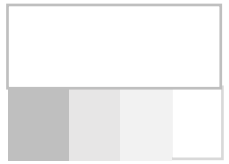
$$normalized = \frac{x - min}{max - \min} \times 255$$

*

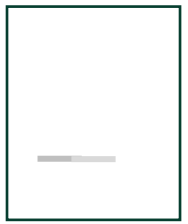
-1	-2	-1
0	0	0
1	2	1

=

0	0	0	0
0	0	0	0
-36	-27	-9	0
-36	-27	-9	0



$$\begin{bmatrix} 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 \\ 0 & 64 & 191 & 255 \\ 0 & 64 & 191 & 255 \end{bmatrix}$$



PyTorch code for applying Sobel filter

```python
import torch
import torch.nn.functional as F

# Define a 2D grayscale image (for simplicity, a 5x5 example)
image = torch.tensor([[10, 20, 30, 40, 50],
                      [15, 25, 35, 45, 55],
                      [20, 30, 40, 50, 60],
                      [25, 35, 45, 55, 65],
                      [30, 40, 50, 60, 70]], dtype=torch.float32)

# Reshape the image to match PyTorch's input format for conv2d: (Batch, Channel, Height, Width)
image = image.unsqueeze(0).unsqueeze(0)  # Shape: (1, 1, 5, 5)

# Define the Sobel filter for horizontal edge detection
sobel_horizontal = torch.tensor([[-1, 0, 1],
                                 [-2, 0, 2],
                                 [-1, 0, 1]], dtype=torch.float32)

# Reshape the Sobel filter to match the conv2d weight format: (Out_channels, In_channels, Height, Width)
sobel_horizontal = sobel_horizontal.unsqueeze(0).unsqueeze(0)  # Shape: (1, 1, 3, 3)

# Apply the Sobel filter using F.conv2d
output_horizontal = F.conv2d(image, sobel_horizontal, padding=1)  # Padding to keep the output size the same

print("Input Image:\n", image.squeeze().numpy())
print("\nHorizontal Edge Detection Output:\n", output_horizontal.squeeze().detach().numpy())
```

Other Hand-Engineered Filters

- **Other Edge detection**: *Prewitt, Scharr* and *Laplacian* kernels

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Prewitt Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Laplacian Kernel

$$\begin{bmatrix} 3 & 0 & -3 \\ 10 & 0 & -10 \\ 3 & 0 & -3 \end{bmatrix}$$

Scharr Kernel

- **Blurring**: *Gaussian* kernel $\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$

- **Texture and edge detection in different orientations** :*Gabor* Kernel

The Challenges of Hand-Engineering Kernels

- **Manual Design**: requires domain expertise and choosing the right filter for an application involves trial and error.
- **Limited Generalizability**: kernels are **fixed** and cannot adapt to different tasks
- **Limited capabilities: work** well for general low-level features (e.g., edges, corners) but struggle with more complex or high-level features.
- **Inefficient** : Hand-engineering requires combining multiple filters to extract different types of features (edges, textures, shapes), which can be computationally intensive and suboptimal.

Here comes the CNN!

1. Learnable Kernels

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

2. Parameter sharing

3. Hierarchical Feature Learning:

4. Generalization

5. Efficiency

Convolutional Neural Network (CNN)

- **A Convolutional Neural Network (CNN)** is a special type of neural network designed to process data with a grid-like structure, such as images.
- CNNs automatically learn spatial hierarchies of features from data, making them ideal for *Visual* recognition and computer vision tasks (preserves spatial structure).
- CNN has been also applied to a wide range of ***NLP*** applications including *sentence classification*, *event detection*, *entity typing*, etc.
- **Why CNNs for Image Processing?**
  - **Images as Grids**: Images can be thought of as grids of pixels. Each pixel has a value (e.g., intensity for grayscale images or RGB values for color images).
  - Manually extracting relevant features from images is difficult and time-consuming.
  - CNNs automatically learn the important features (edges, textures, shapes) at different scales, reducing the need for manual feature engineering.

Convolutional Neural Network (CNN)

- Deep CNN architecture take an input (typically a image) and , pass it through a series of
  - Convolutional layers
  - Nonlinear activation function (typically ReLU)
  - pooling (down-sampling)
  - fully connected layers and loss function (e.g. max-margin/Softmax) on top.

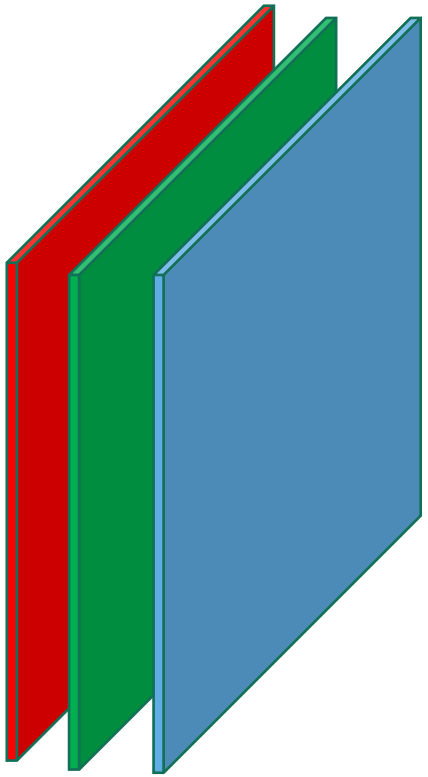
Convolutional Neural Network (CNN)

- The forward phase in CNN is more efficient than it is in feed-forward NN, as CNN has way less parameters.
- In regular NN, every neuron from a layer is fully connected to every neuron from the next layer.
- The full connectivity in feedforward NN → a huge number of parameters → overfitting
  - **For Example,** In a binary 2-level neural network, using a layer with 100 hidden node to read a 256×256 RGB image (3 channels) requires learning $256 \times 256 \times 3 \times 100 + 100$ (bias) parameters need to learn 19,660,900 parameters→ $way\ too\ many$

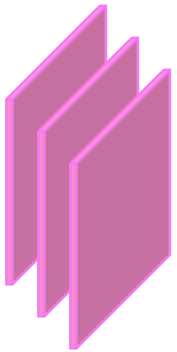
Convolutional Layer

- CNN makes assumption input are images.
- Images has height, width and depth e.g., $32 \times 32 \times 3$ image.
- **Filter**(aka **kernel**) slides (or **convolve) spatially** over the **i**nput image region by region and compute dot product.
- The region on the image that is multiplied with the filter is is called **receptive field**
- The dot product is computed by **element-wise multiplying the filter by the receptive field then the** multiplications are all summed up to produce a single number.
- The filter which is a matrix of numbers represent **weights** or **parameters** to be learned from the data.
- The output of the convolution operation is called **Activation map or feature map.**

Convolutional Layer



$8 \times 8 \times 3$ Image



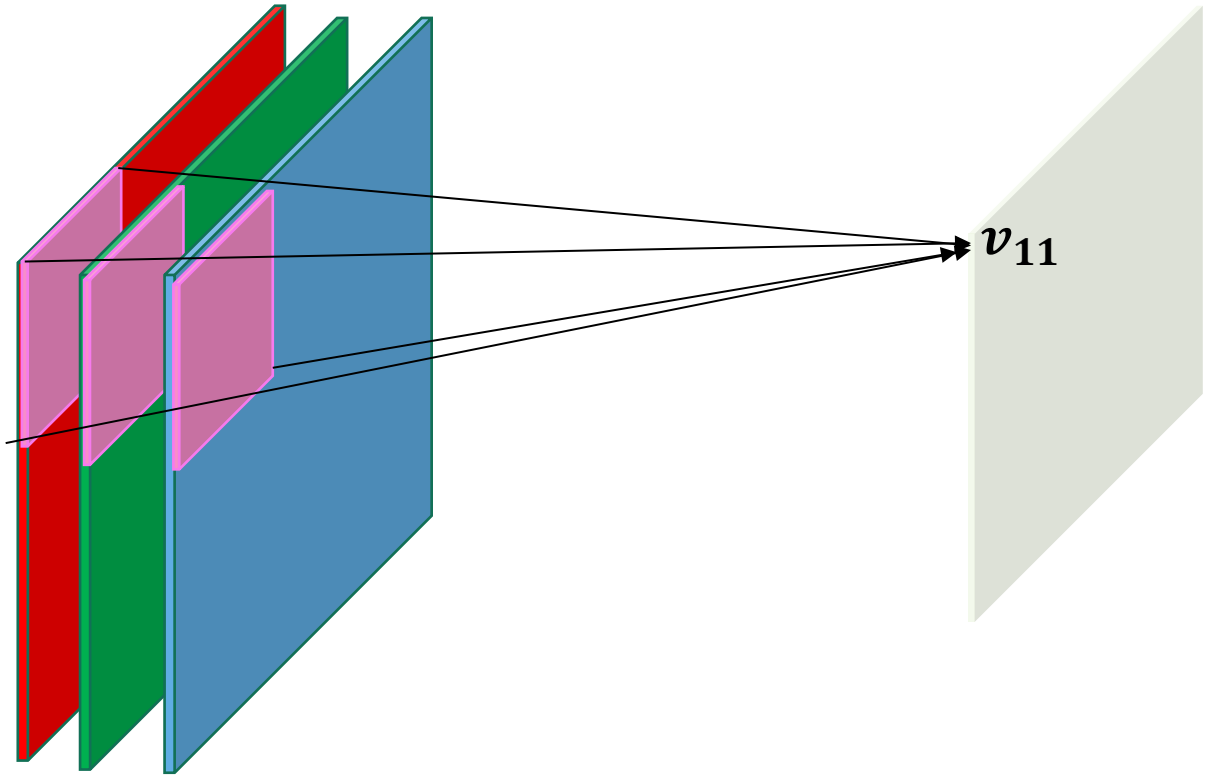
Red Filter
Green Filter
Blue Filter

$4 \times 4 \times 3$ Filter

Each filter is associated to 1 channel

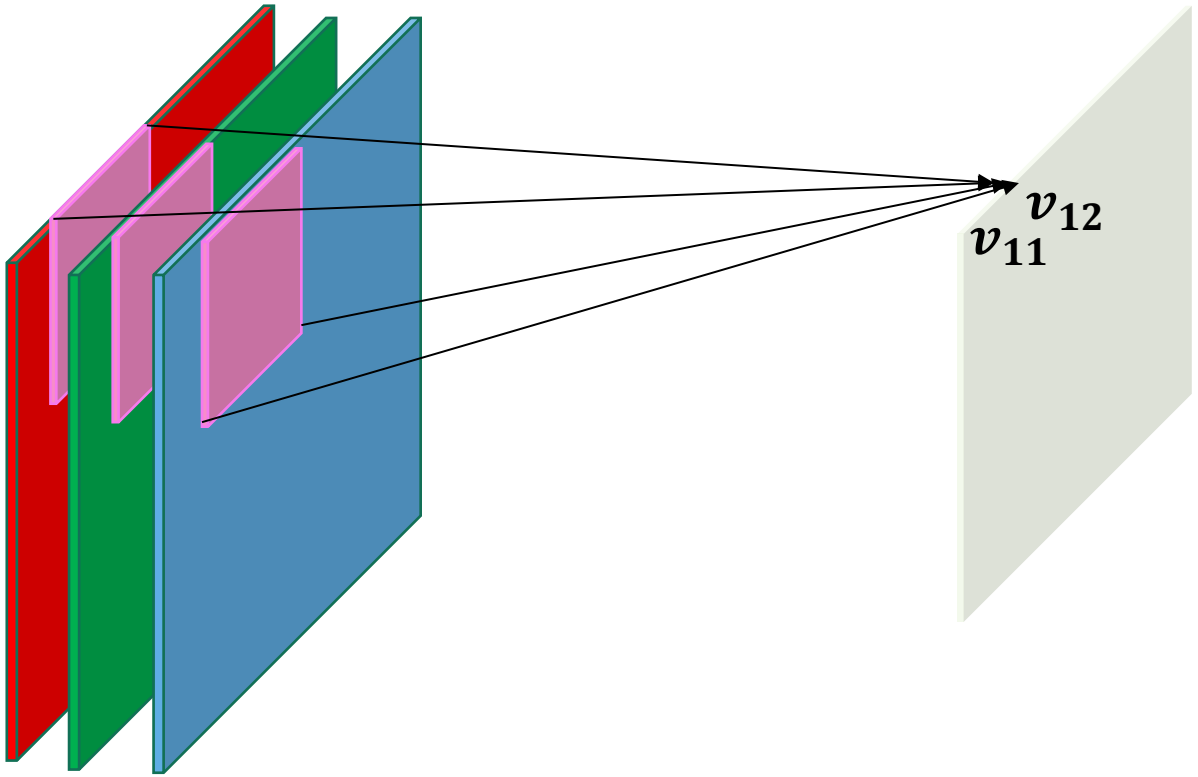
The depth of the filter is the same as the depth of the input

Convolutional Layer



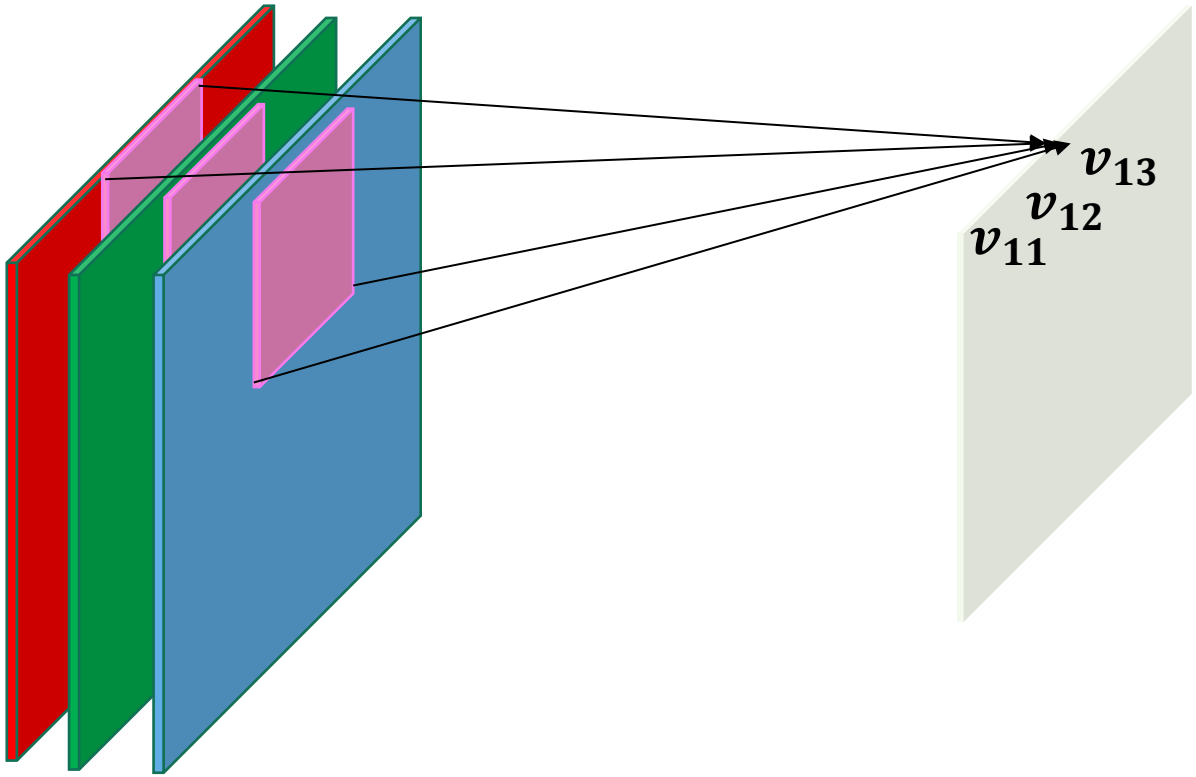
Multiply the filter by $8 \times 8 \times 3$ region of the Image and sum the multiplication, and add a bias, the results is a single value

Convolutional Layer



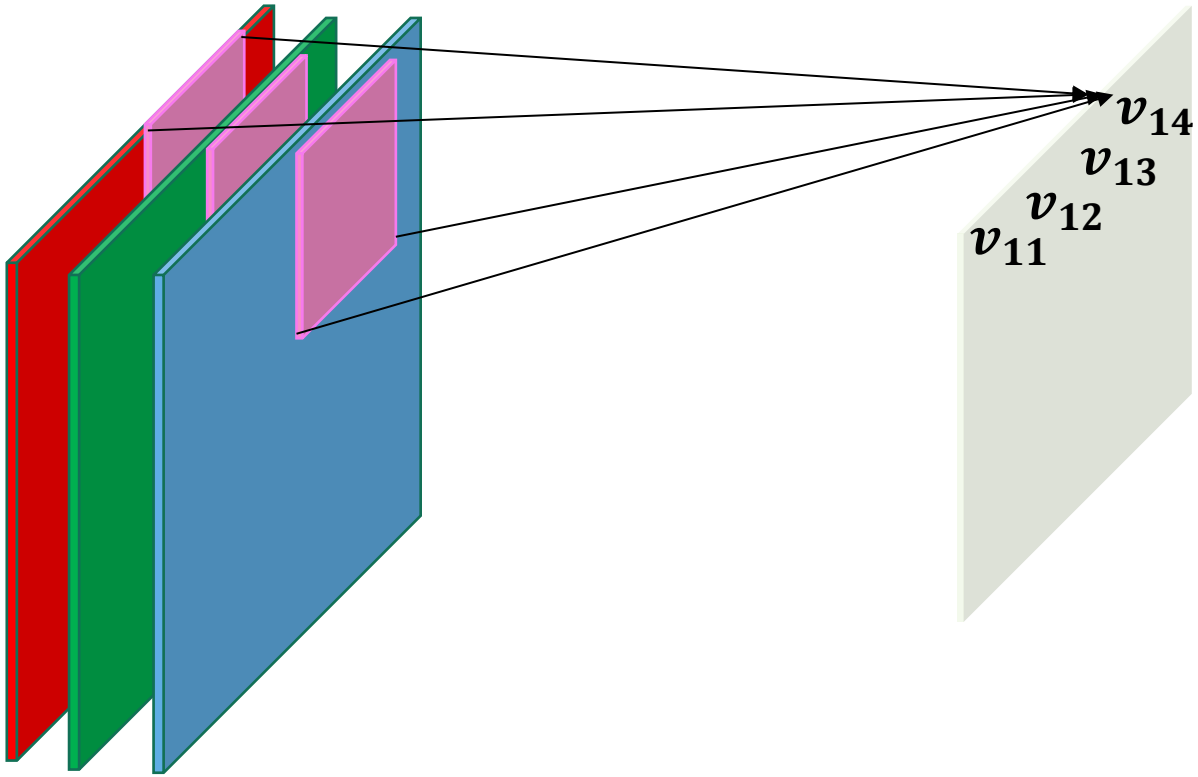
Multiply the filter by $8 \times 8 \times 3$ region of the Image and sum the multiplication, and add a bias, the results is a single value

Convolutional Layer



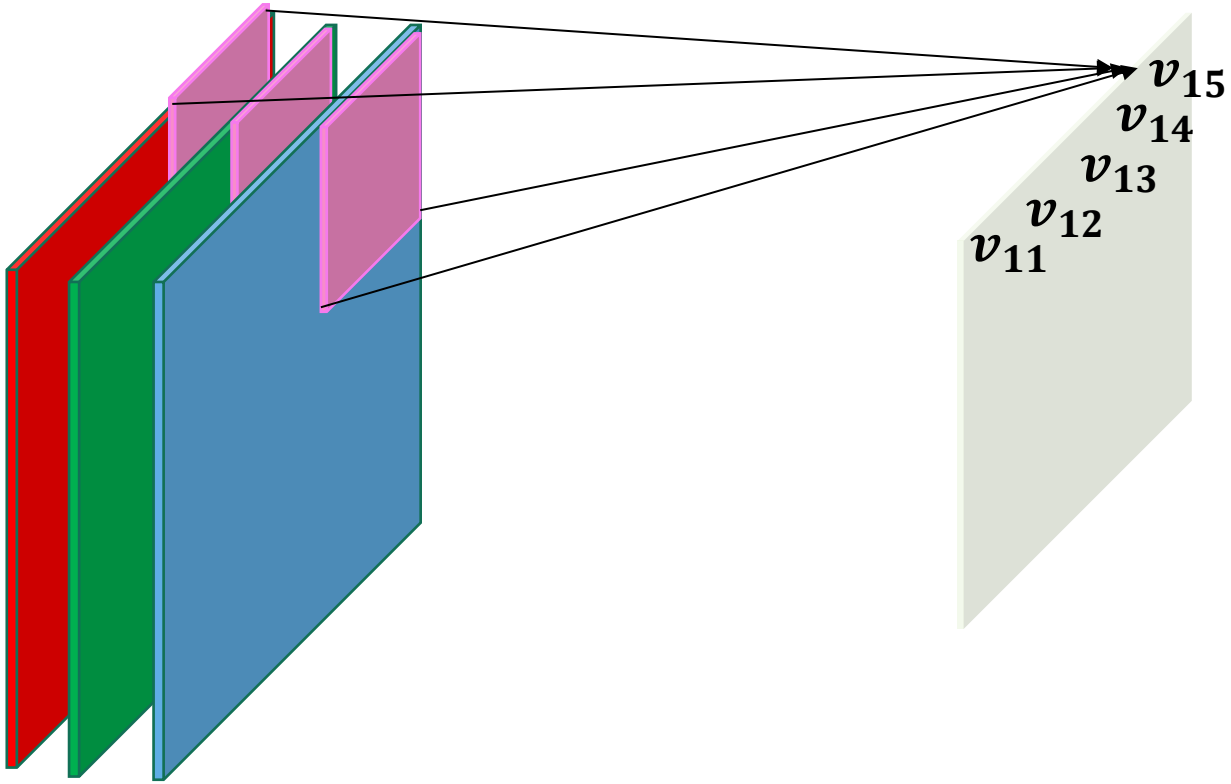
Multiply the filter by $8 \times 8 \times 3$ region of the Image and sum the multiplication, and add a bias, the results is a single value

Convolutional Layer



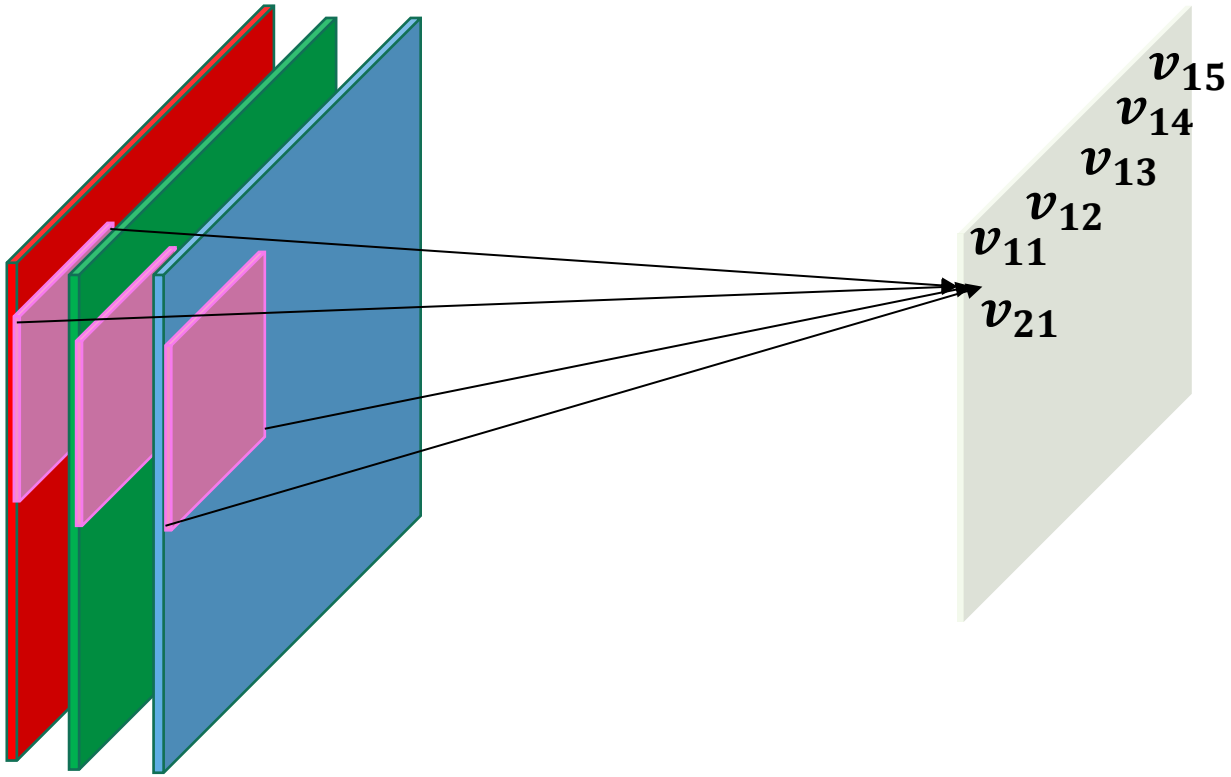
Multiply the filter by $8 \times 8 \times 3$ region of the Image and sum the multiplication, and add a bias, the results is a single value

Convolutional Layer



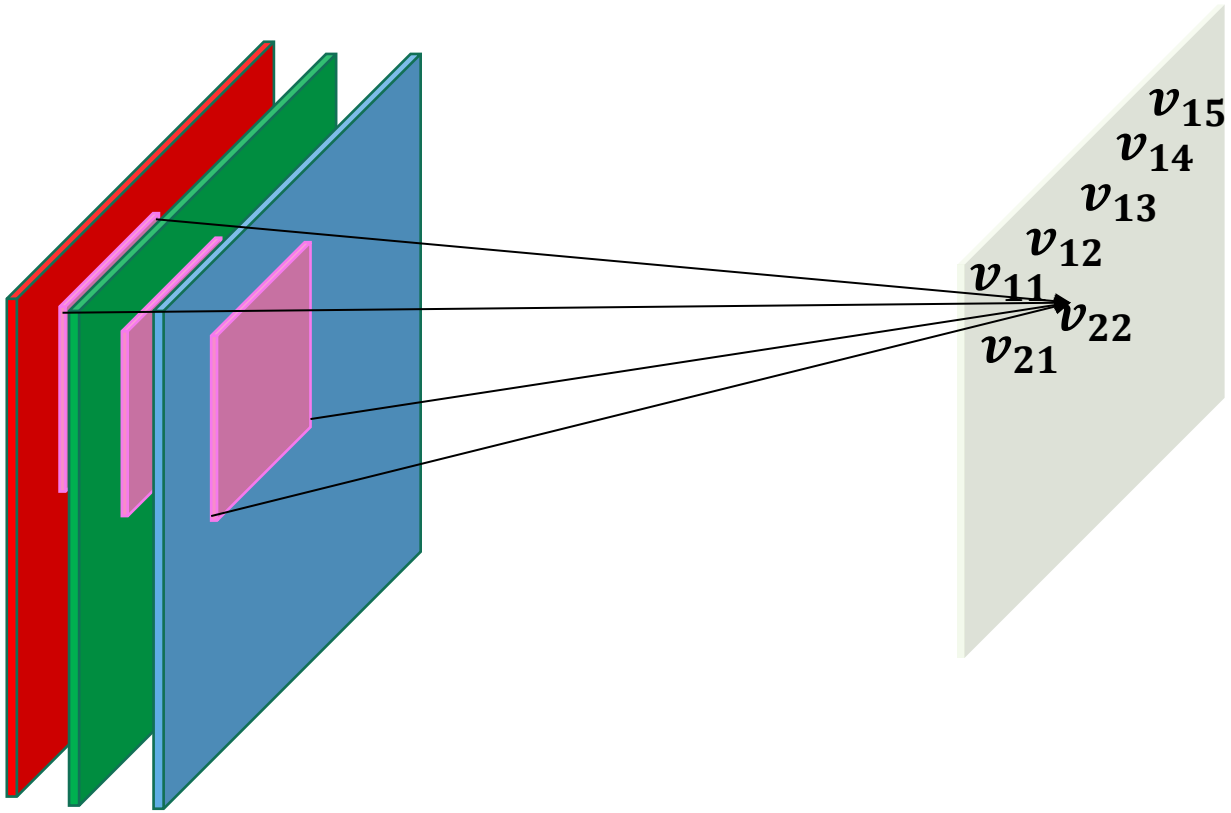
Multiply the filter by $8 \times 8 \times 3$ region of the Image and sum the multiplication, and add a bias, the results is a single value

Convolutional Layer



Multiply the filter by $8 \times 8 \times 3$ region of the Image and sum the multiplication, and add a bias, the results is a single value

Convolutional Layer



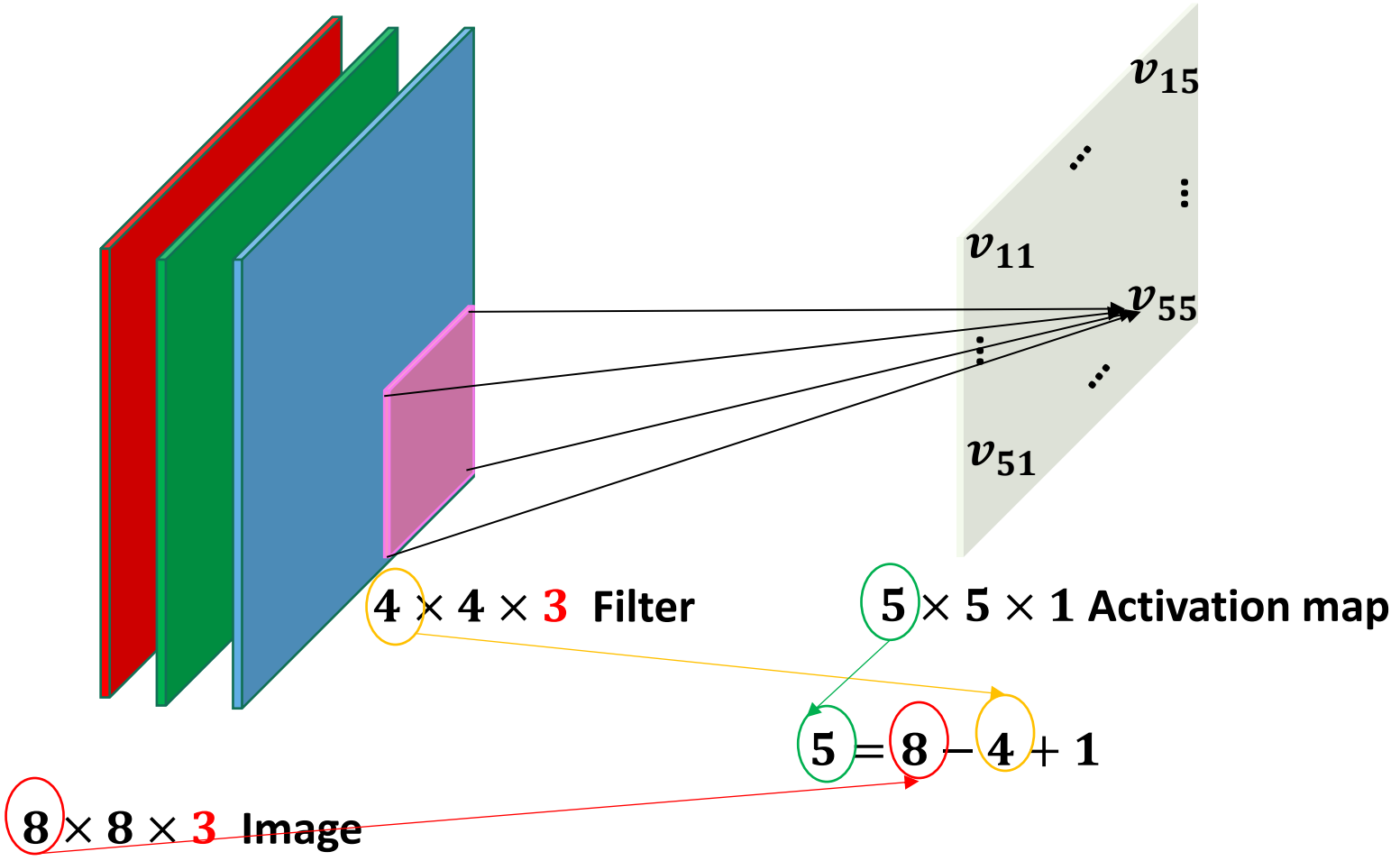
Multiply the filter by $8 \times 8 \times 3$ region of the Image and sum the multiplication, and add a bias, the results is a single value

Convolutional Layer

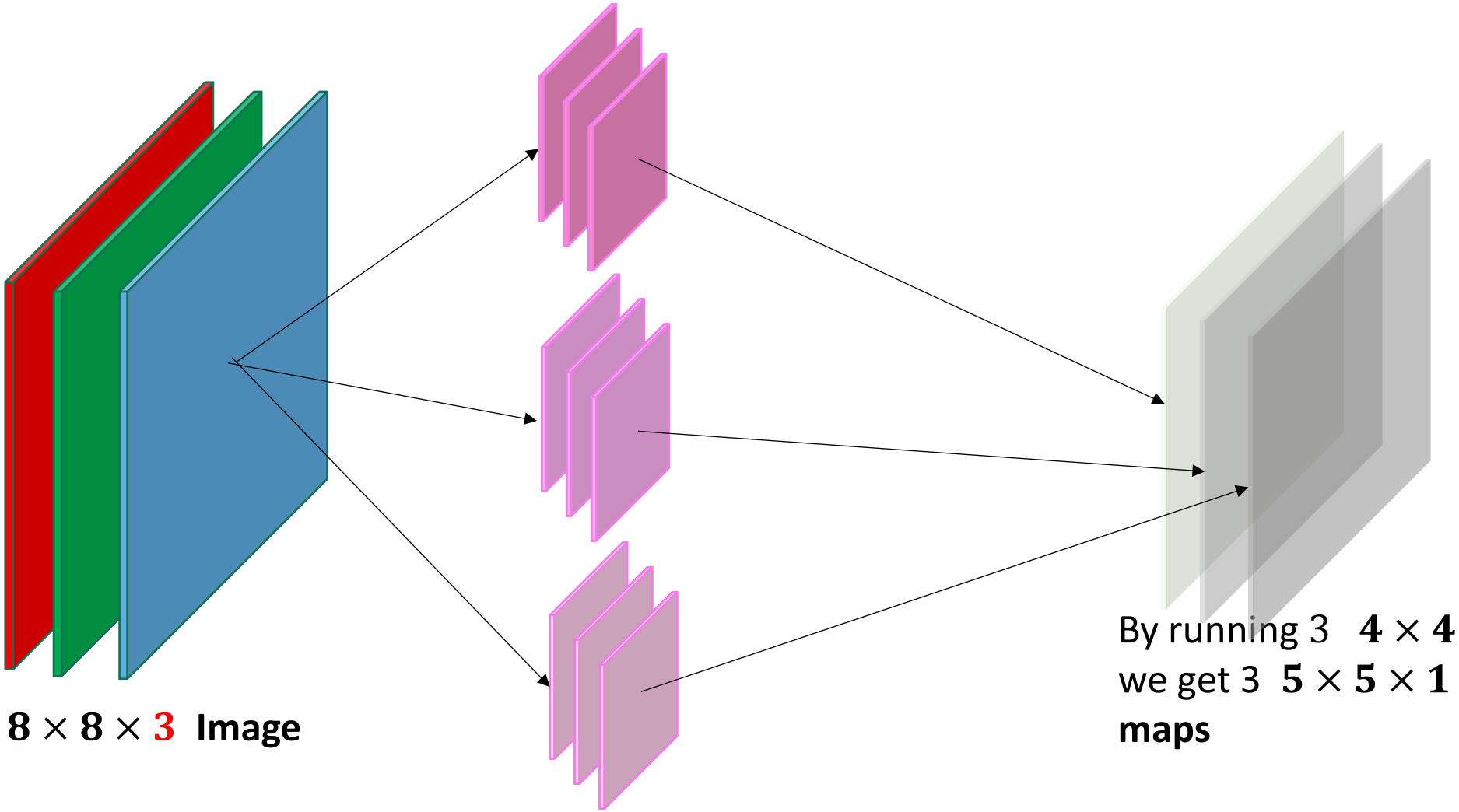
Keep convolving until convolving over the whole image…….

Convolutional Layer

v_{ij}: is called neuron or activation



Convolutional Layer

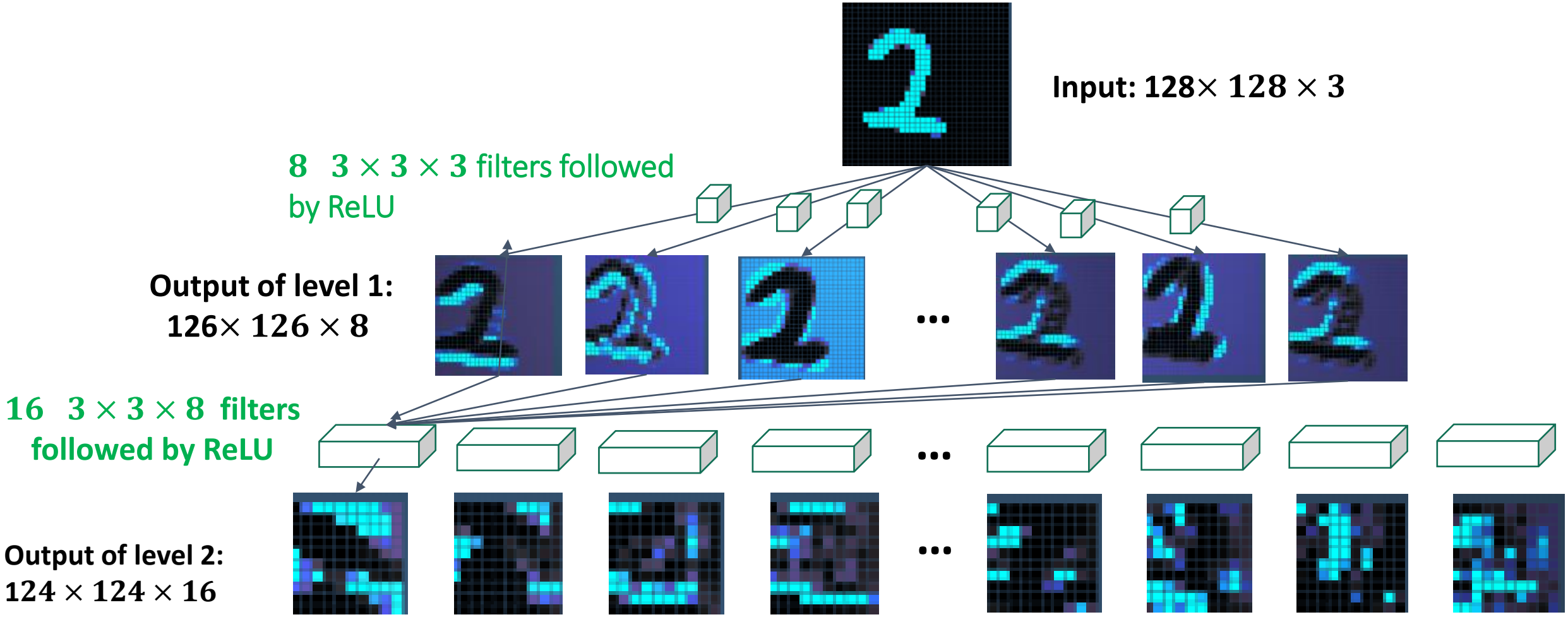


$8 \times 8 \times 3$ **Image**

$4 \times 4 \times 3$ **Filter**

By running 3 $4 \times 4 \times 1$ **filters,** we get 3 $5 \times 5 \times 1$ **activation maps**

Filter and Output Shapes



Advantage of Parameter sharing and Sparsity of Connections in CNNs

❖ **Sparsity of Connections:** in CNNs, each **neuron** in a layer is connected to only a small, localized region of the input (i.e., **receptive field**). This is different from fully connected layers, where every neuron is connected to every input pixel.

❖ **Parameter sharing** refers to the idea that a filter (or kernel) used to detect a feature (e.g., a vertical edge) is applied ***across all spatial locations*** of an image. This means the same filter is "shared" throughout the entire image.

❖ Example:

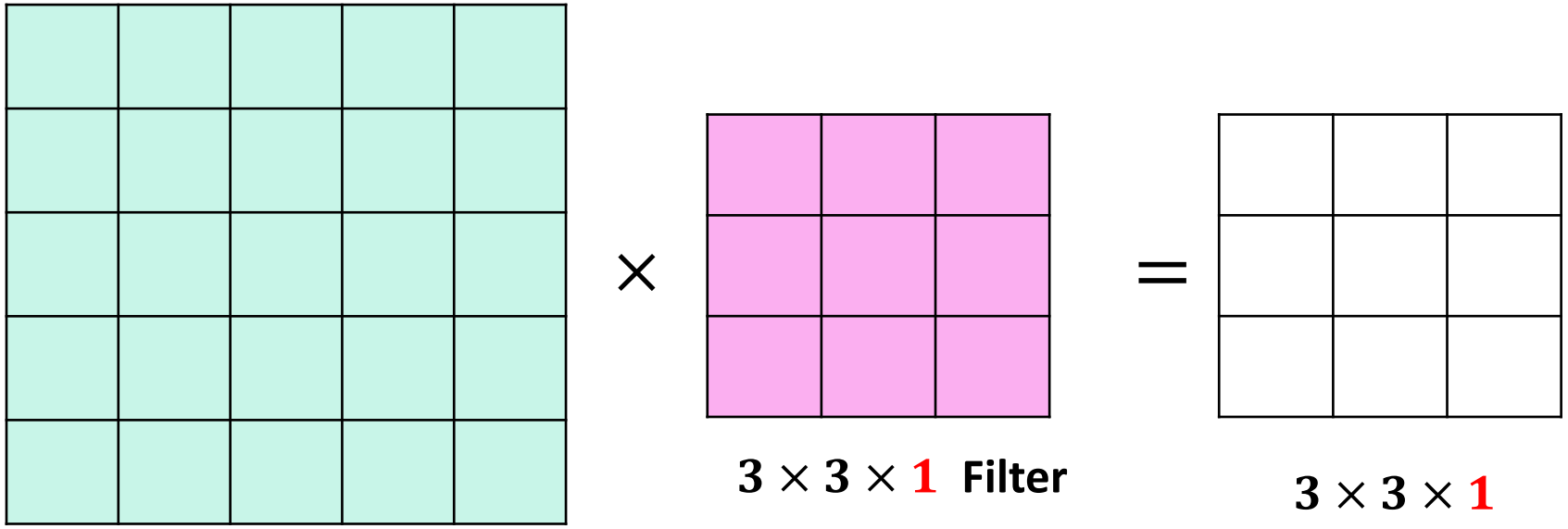
- In a binary 2-level neural network, using a layer with 100 hidden node to read a 256×256 RGB image (3 channels) requires learning $256 \times 256 \times 3 \times 100 + 100$ (bias) parameters need to learn 19,660,900 parameters→ $way\ too\ many$
- In CNN , using **one Convolutional layer 100 filters**, each of size **5×5** and operating on a **256×256 RGB image**. The number of learnable parameters in T**his convolutional layer** is computed as (100× 5×5×3) +100 (bias term for each filter) =7600 → A huge reduction on the number of parameters to be learned!

Stride

- **Stride** is the number of positions the filter shifts over the input matrix.
- It is a hyper-parameter.
- Can be used to down-sample the size of the output activation map.

Stride

Convolution with Stride 1



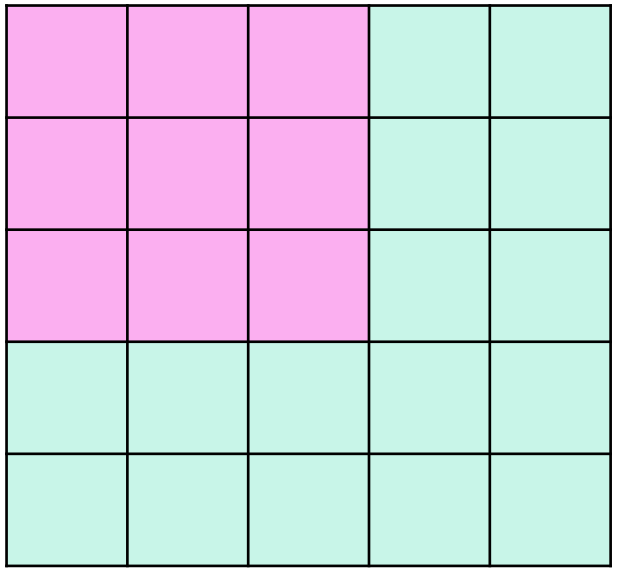
$5 \times 5 \times 1$ **Image**

$3 \times 3 \times 1$ **Filter**

$3 \times 3 \times 1$
activation maps

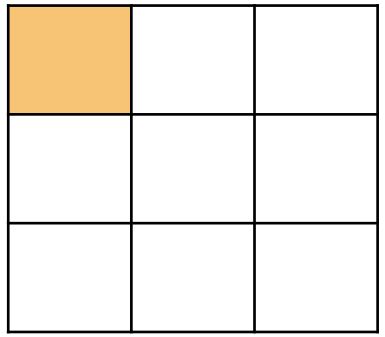
Stride

Convolution with Stride 1



$5 \times 5 \times 1$ **Image**

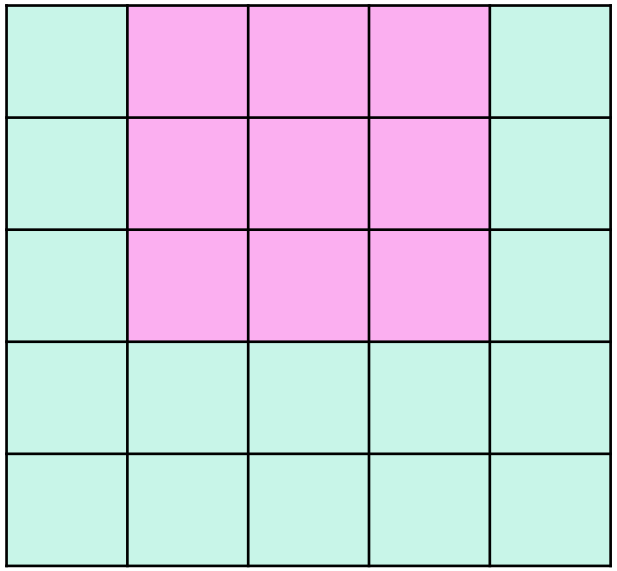
=



$3 \times 3 \times 1$
activation maps

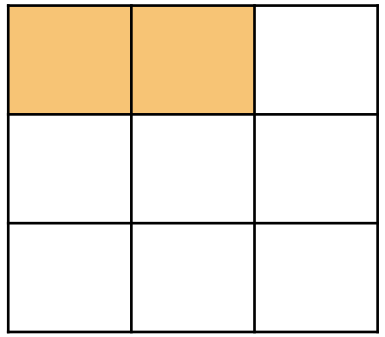
Stride

Convolution with Stride 1



$5 \times 5 \times 1$ **Image**

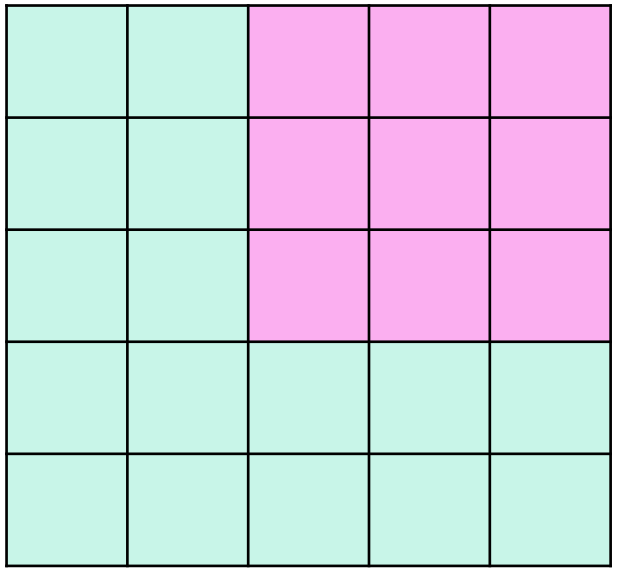
=



$3 \times 3 \times 1$
activation maps

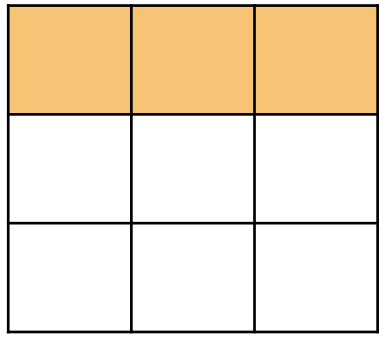
Stride

Convolution with Stride 1



$5 \times 5 \times 1$ Image

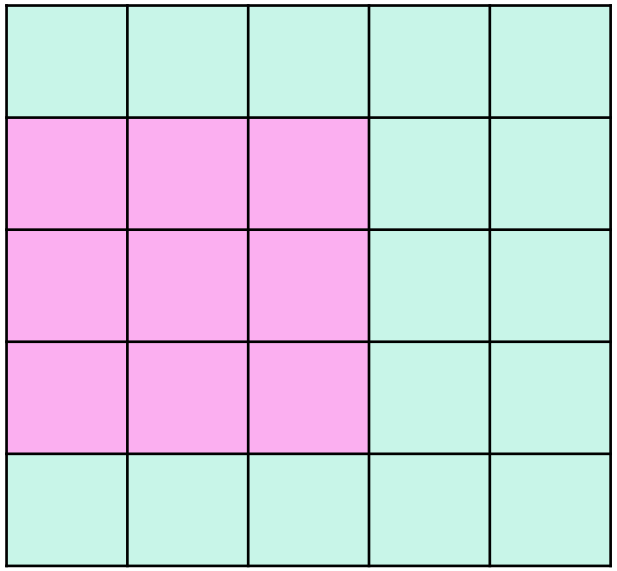
=



$3 \times 3 \times 1$
activation maps

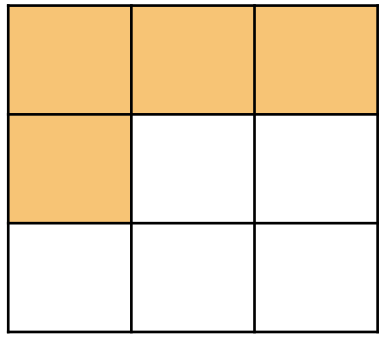
Stride

Convolution with Stride 1



$5 \times 5 \times 1$ **Image**

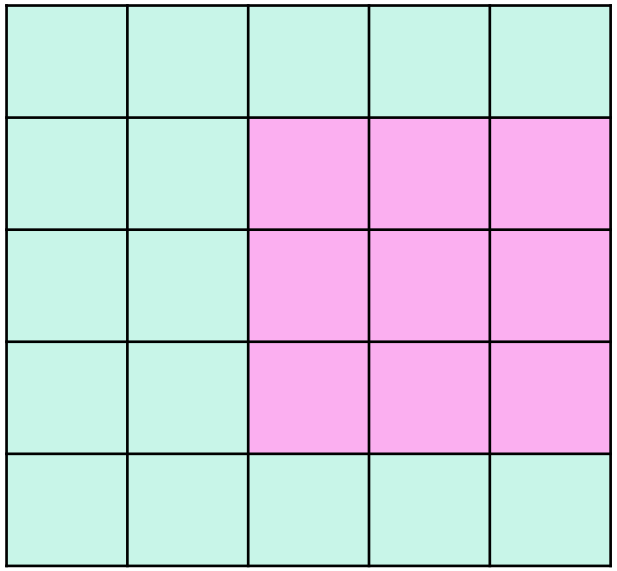
=



$3 \times 3 \times 1$
activation maps

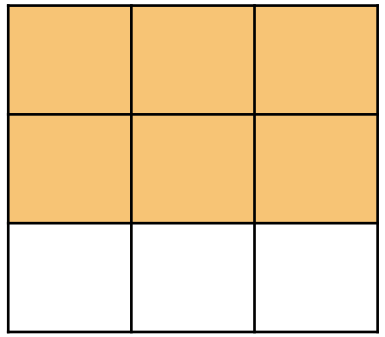
Stride

Convolution with Stride 1



$5 \times 5 \times 1$ **Image**

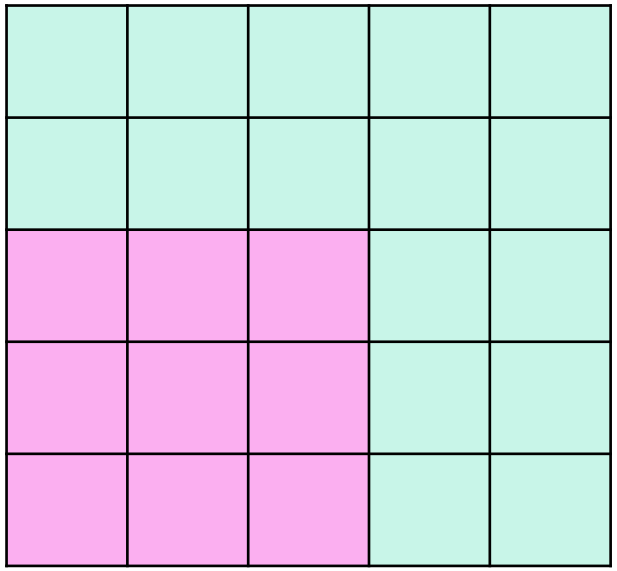
=



$3 \times 3 \times 1$
activation maps

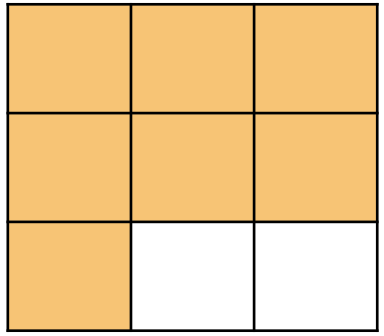
Stride

Convolution with Stride 1



$5 \times 5 \times 1$ **Image**

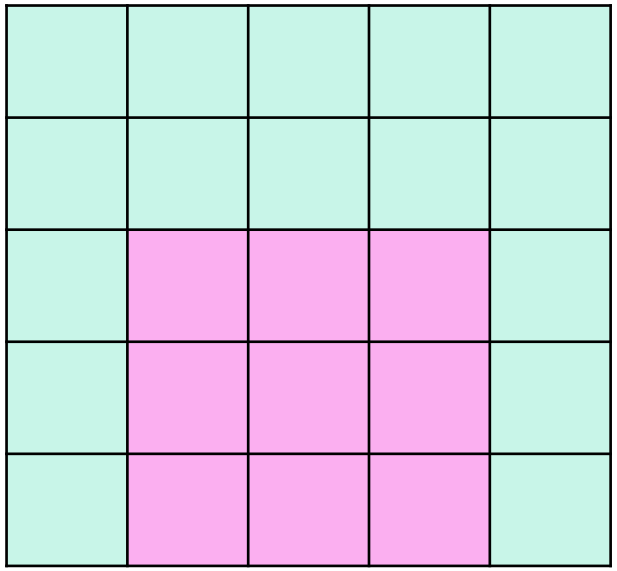
=



$3 \times 3 \times 1$
activation maps

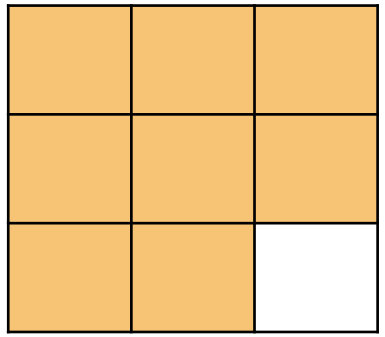
Stride

Convolution with Stride 1



$5 \times 5 \times 1$ **Image**

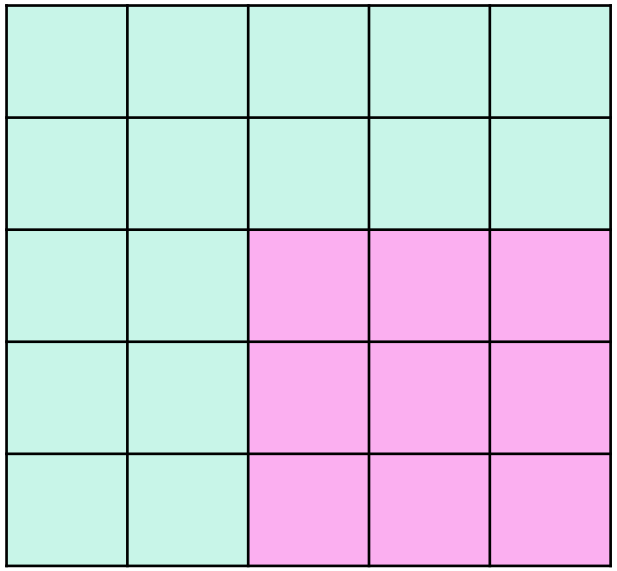
=



$3 \times 3 \times 1$
activation maps

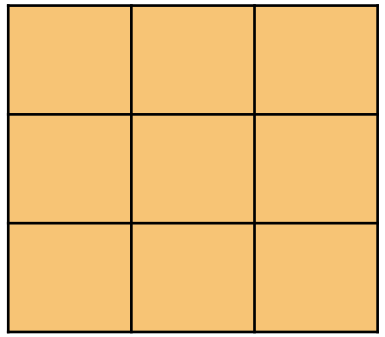
Stride

Convolution with Stride 1



$5 \times 5 \times 1$ **Image**

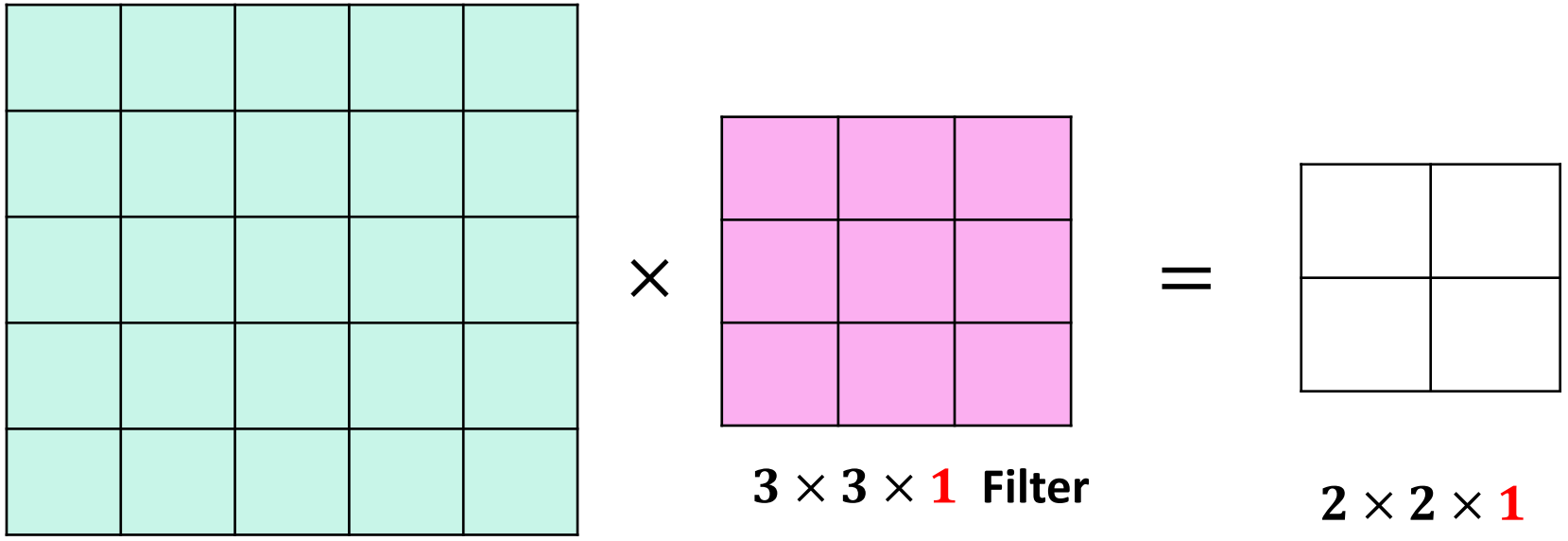
=



$3 \times 3 \times 1$
activation maps

Stride

Convolution with Stride 2



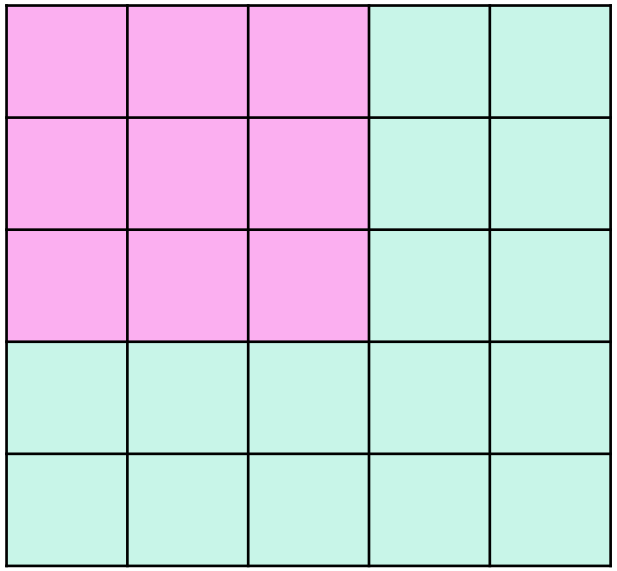
$5 \times 5 \times 1$ **Image**

$3 \times 3 \times 1$ **Filter**

$2 \times 2 \times 1$
activation maps

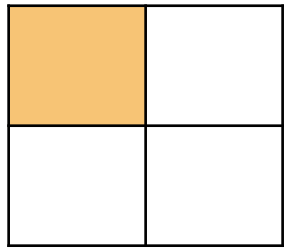
Stride

Convolution with Stride 2



$5 \times 5 \times 1$ **Image**

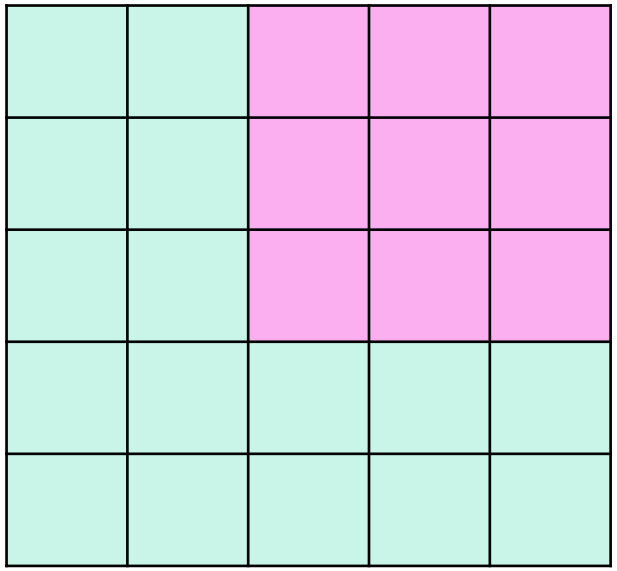
=



$2 \times 2 \times 1$
activation maps

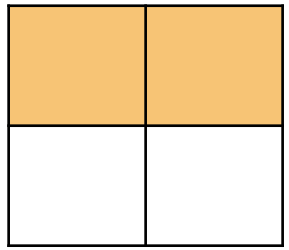
Stride

Convolution with Stride 2



$5 \times 5 \times 1$ **Image**

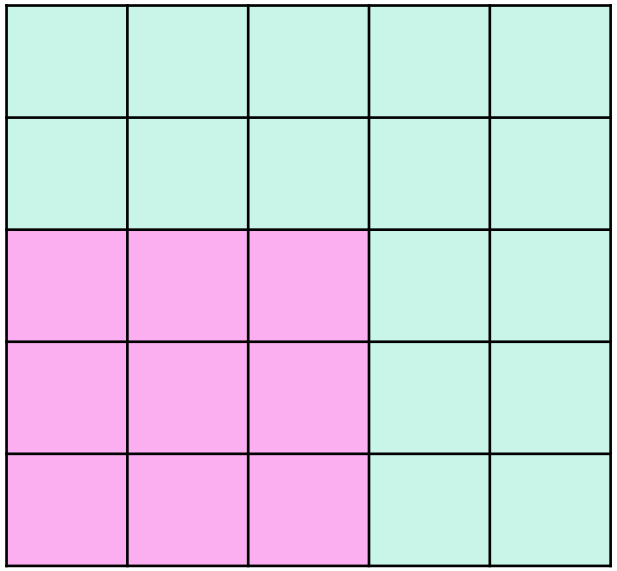
=



$2 \times 2 \times 1$
activation maps

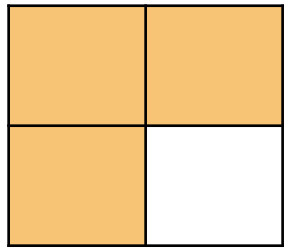
Stride

Convolution with Stride 2



$5 \times 5 \times 1$ **Image**

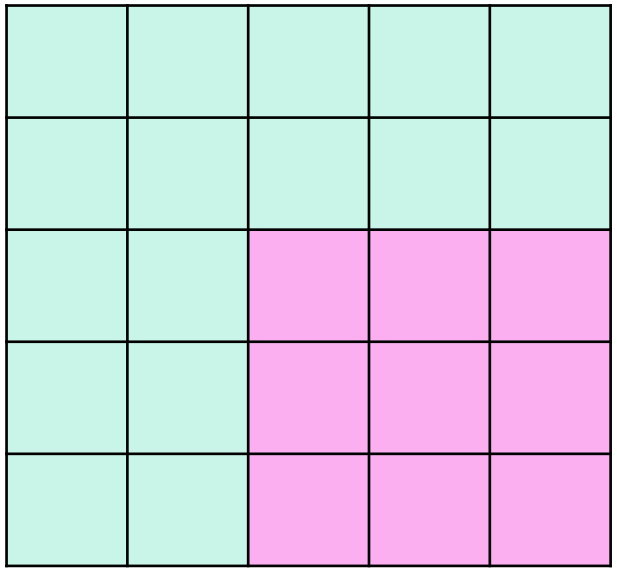
=



$2 \times 2 \times 1$
activation maps

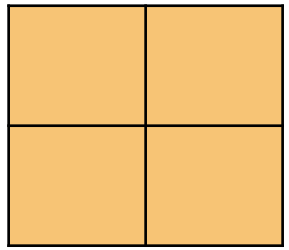
Stride

Convolution with Stride 2



$5 \times 5 \times 1$ **Image**

=



$2 \times 2 \times 1$
activation maps

Why stride

- **Controls output size**: reducing feature map dimensions with larger strides.
- **Improves Computational Efficiency**: Larger strides reduce the number of convolution operations by skipping positions
- **Encourages Location Invariance**: Changing the stride of the convolution across the image helps to achieve location invariance (*features detected are not tied to specific pixel positions*).
- **Receptive Field Expansion**: Increasing the stride expands the **receptive field** by allowing each neuron in the feature map to "see" a larger portion of the input, enabling the network to capture more *global patterns and contextual relationships* in the image.

Stride

- Let's assume input is $W^i \times H^i \times ch$ To get the size of the output activation map $W^o \times H^o \times 1$ we use

$$W^o = \left(\frac{W^i - F}{S}\right) + 1$$

$$H^o = \left(\frac{H^i - F}{S}\right) + 1$$

W^i: Width of the input.
H^i: Height of the input.
W^o: Width of the output.
H^o: Height of the output.
F: Filter size
S: stride

- Where F is the filter size.
- For example,
  - $5 \times 5 \times 1$ Image, $3 \times 3 \times 1$ Filter, S =1 $\rightarrow W^o = H^o = ((5-3)/1) + 1 = 3$

  $\rightarrow$ the output is $3 \times 3 \times 1$ activation map
  - $5 \times 5 \times 1$ Image, $3 \times 3 \times 1$ Filter, S =2 $\rightarrow W^o = H^o = ((5-3)/2) + 1 = 2$

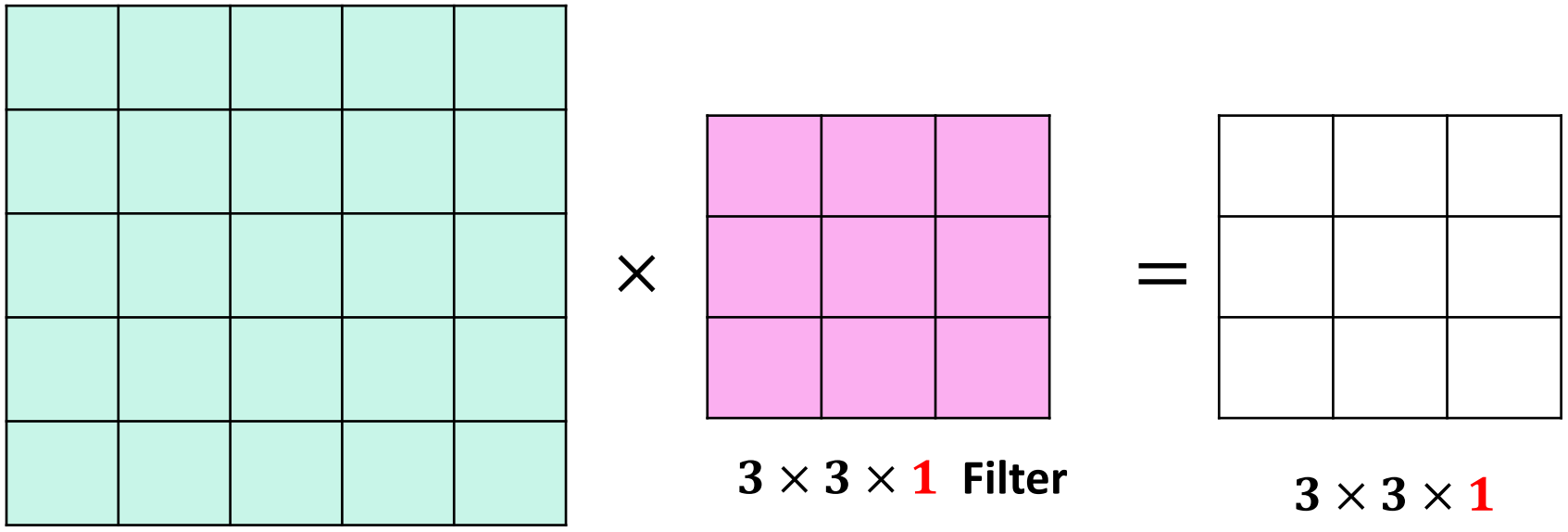
  $\rightarrow$ the output is $2 \times 2 \times 1$ activation map

  If any of W^o or H^o is not an integer value, then padding is needed!

If any of W^o or H^o is not an integer value, it indicates that the filter size and stride are not compatible with the input dimensions (Padding is needed!)

Stride

What about convolution with Stride 3?



$5 \times 5 \times 1$ **Image**

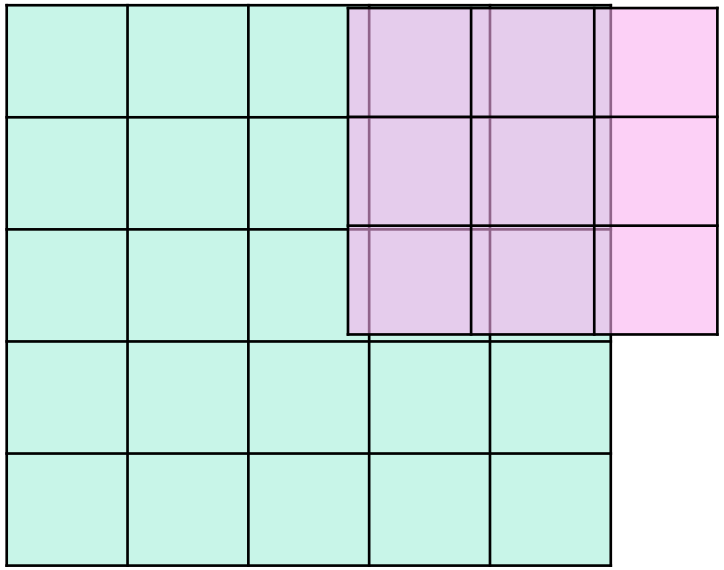
$3 \times 3 \times 1$ **Filter**

$3 \times 3 \times 1$
activation maps

Stride

What about convolution with Stride 3?

It doesn't fit !



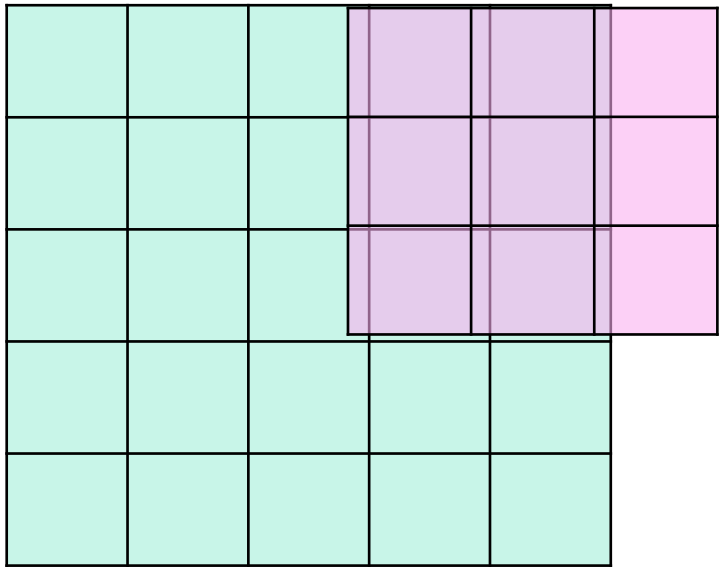
$5 \times 5 \times 1$ Image

Stride

What about convolution with Stride 3?

It doesn’t fit !

Solution ?!



$5 \times 5 \times 1$ Image

Stride with padding

- What about the convolution with Stride 3?

  It doesn't fit !! Solution ?

  Padding: zero pad the border

- Padding is also used to preserve the output size spatially, I.e. prevent the size the output from shrinking too fast in multi-layer ConvNet→not good in practice

Stride with padding

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Pad $5 \times 5 \times 1$ Image with 2-pixel border

Stride with padding

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Pad $5 \times 5 \times 1$ Image with 2 pixel border

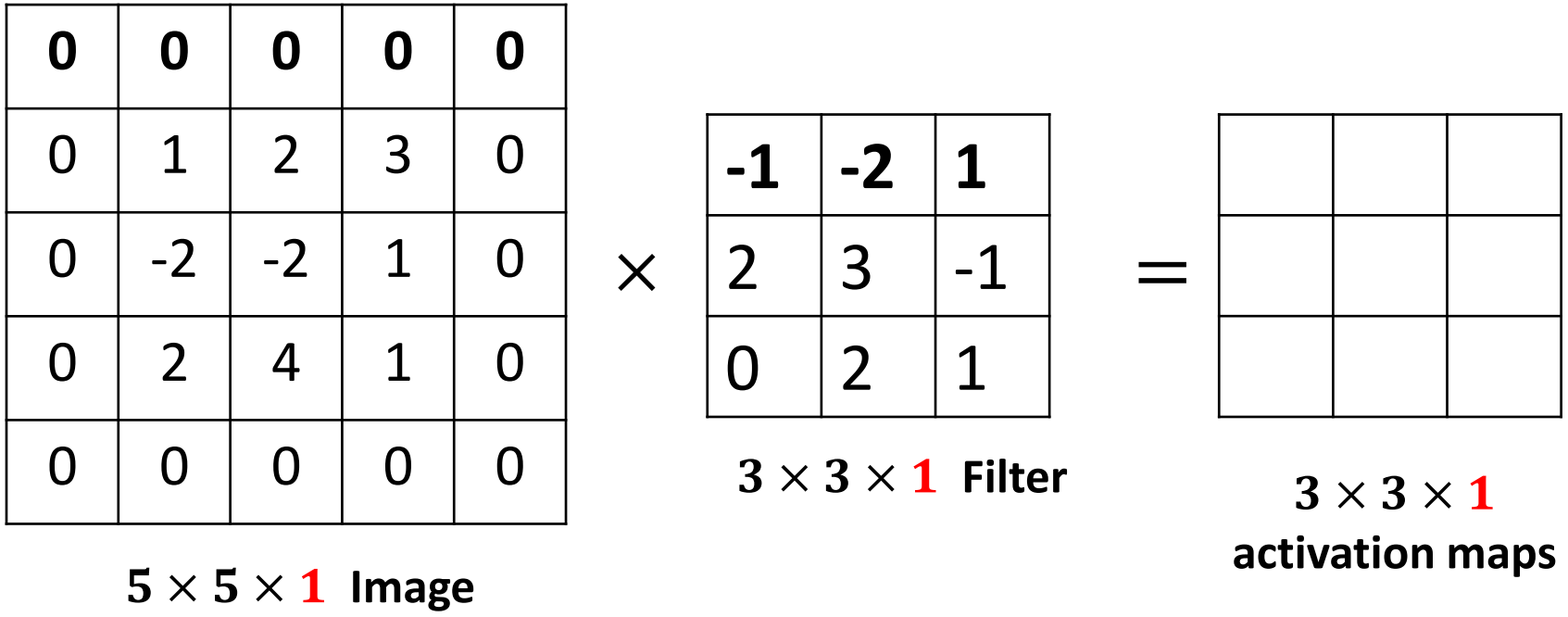
Stride with padding

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Pad $5 \times 5 \times 1$ Image with 2 pixel border

Convolutional Layer

- Example : convolution with padding (stride=1)



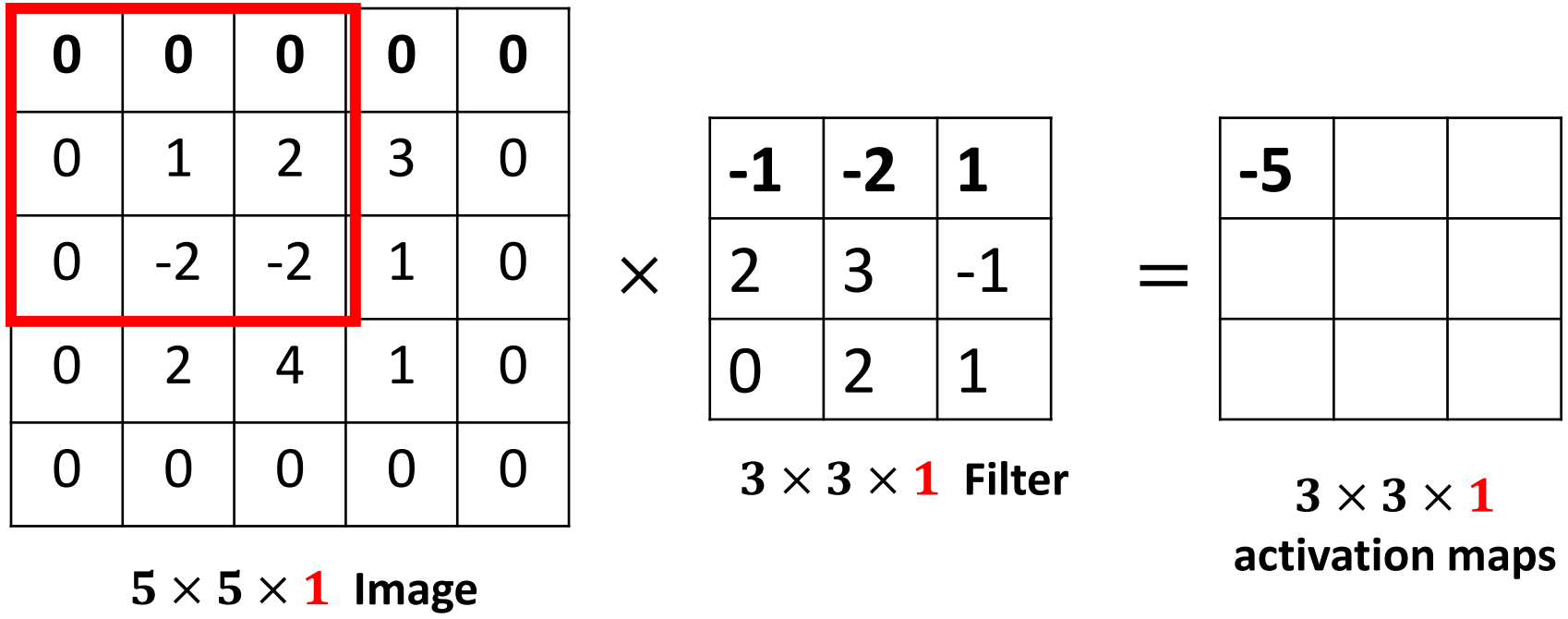
$5 \times 5 \times 1$ **Image**

$3 \times 3 \times 1$ **Filter**

$3 \times 3 \times 1$
activation maps

Convolutional Layer

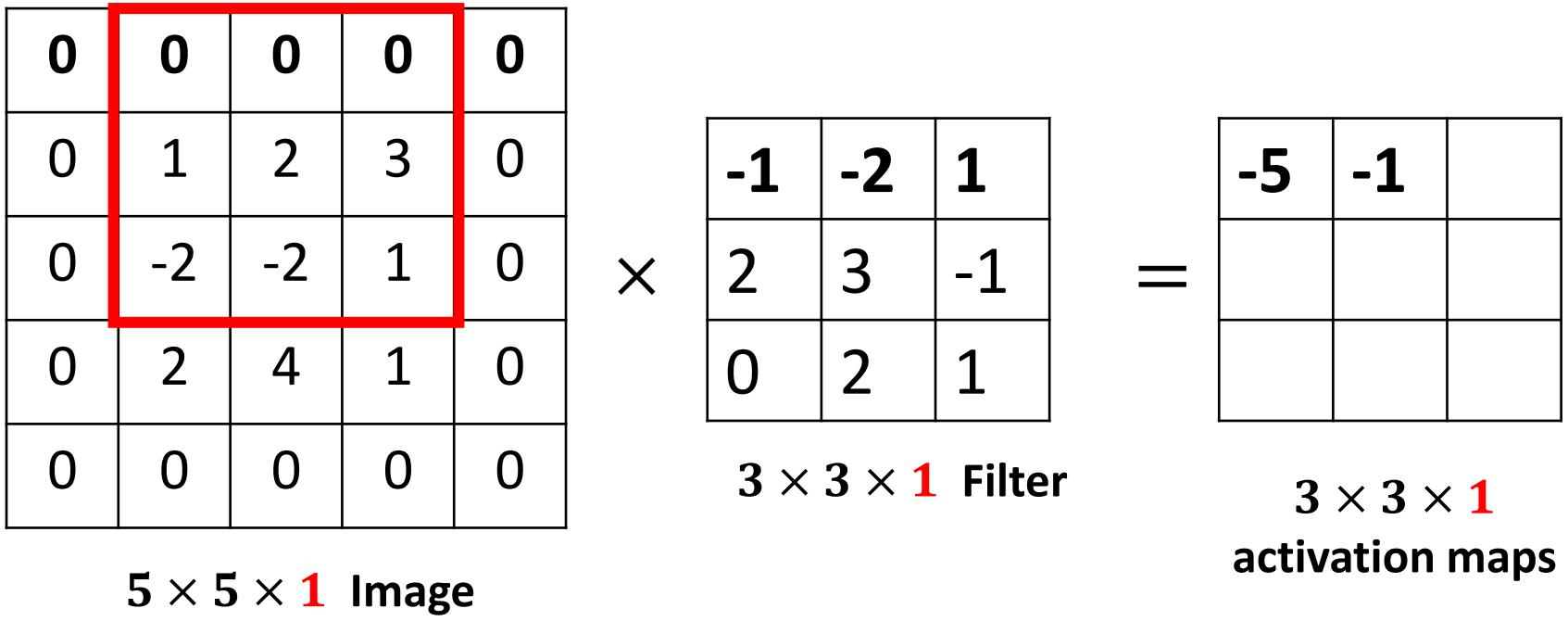
- Example : convolution with padding (stride=1)



$$(1 \times 3) + (2 \times -1) + (-2 \times 2) + (-2 \times 1) = -5$$

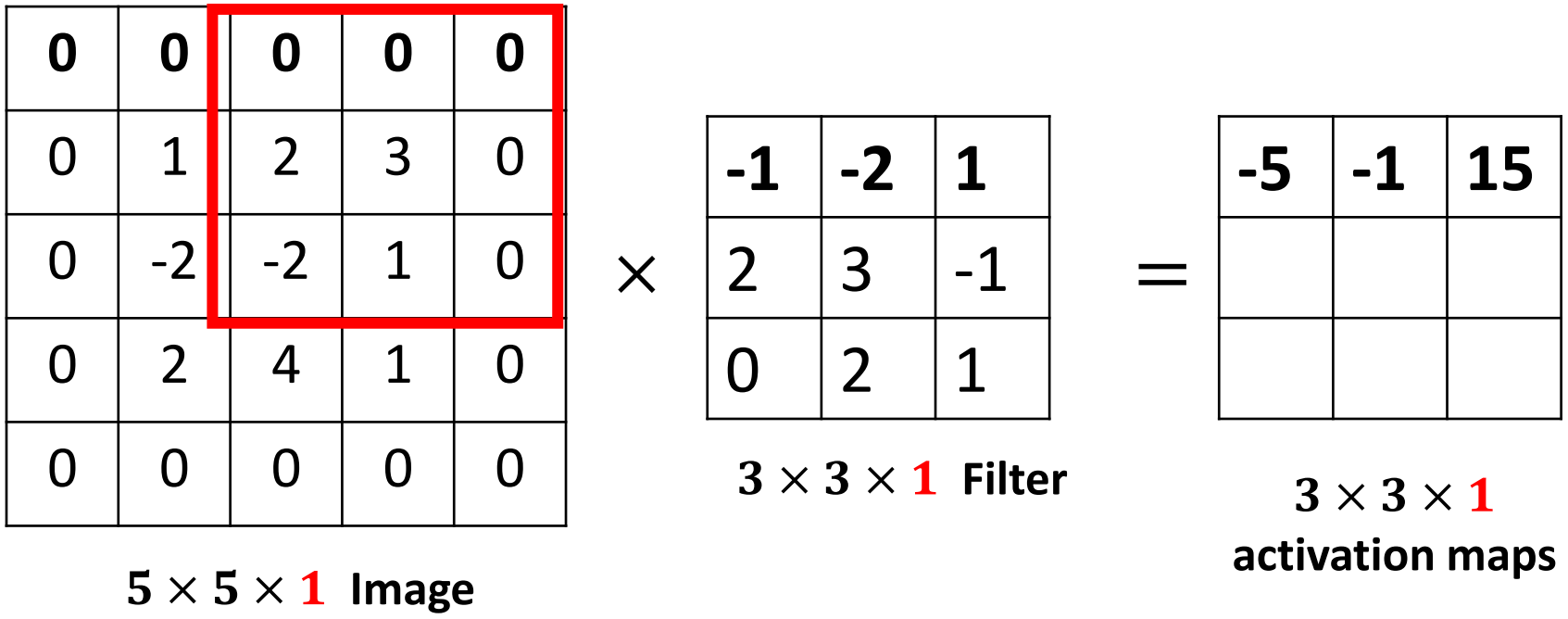
Convolutional Layer

- Example : convolution with padding (stride=1)



Convolutional Layer

- Example : convolution with padding (stride=1)



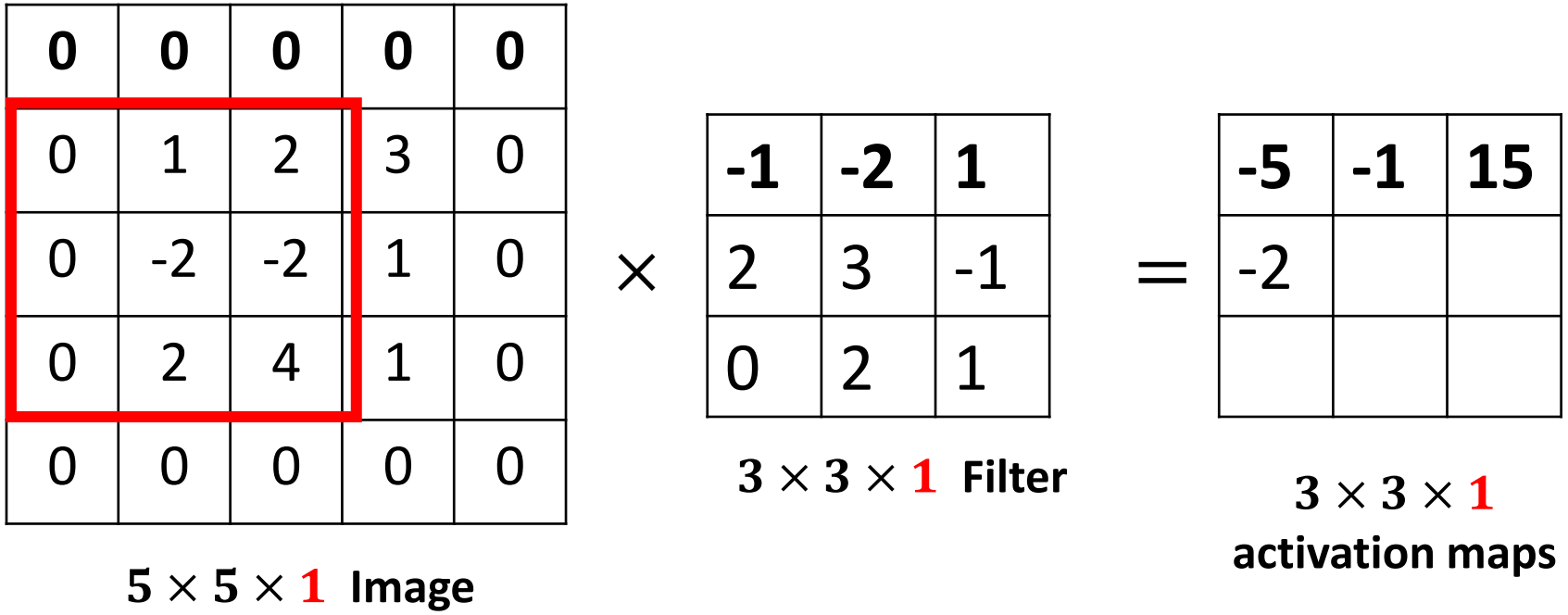
$5 \times 5 \times 1$ **Image**

$3 \times 3 \times 1$ **Filter**

$3 \times 3 \times 1$
activation maps

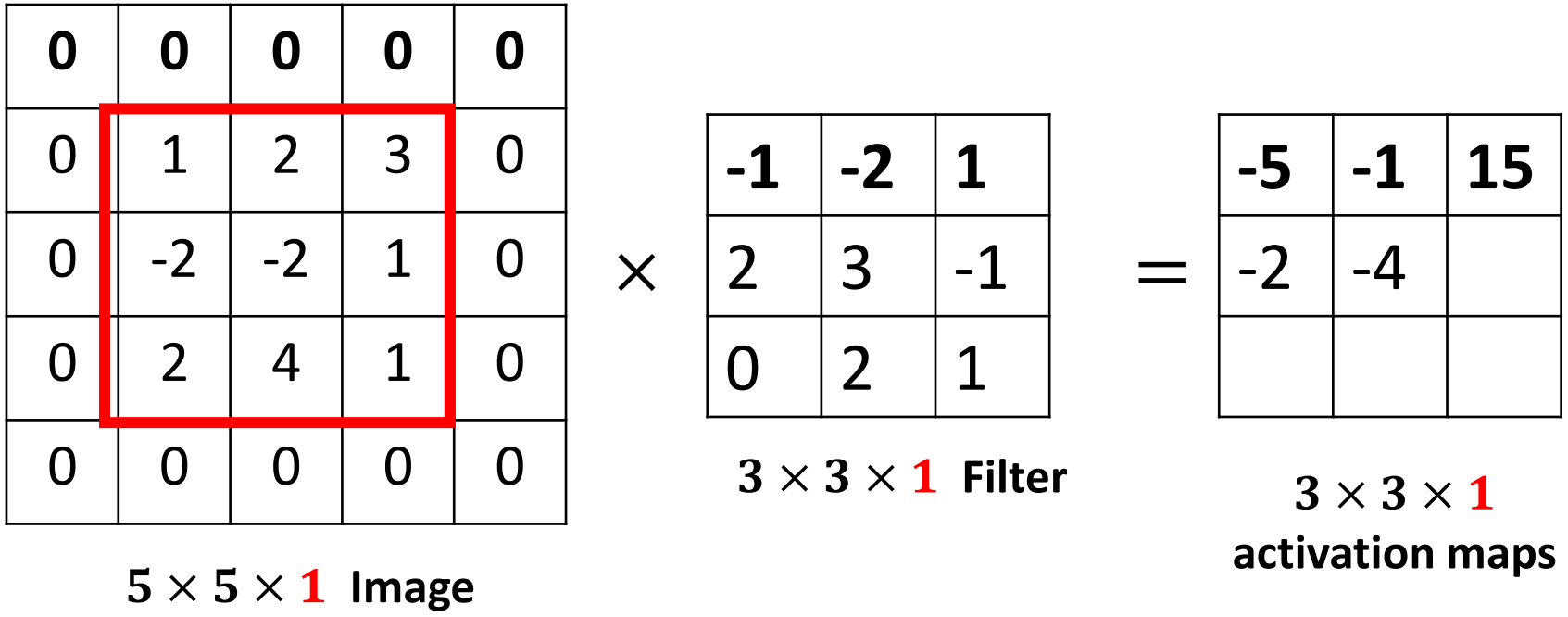
Convolutional Layer

- Example : convolution with padding (stride=1)



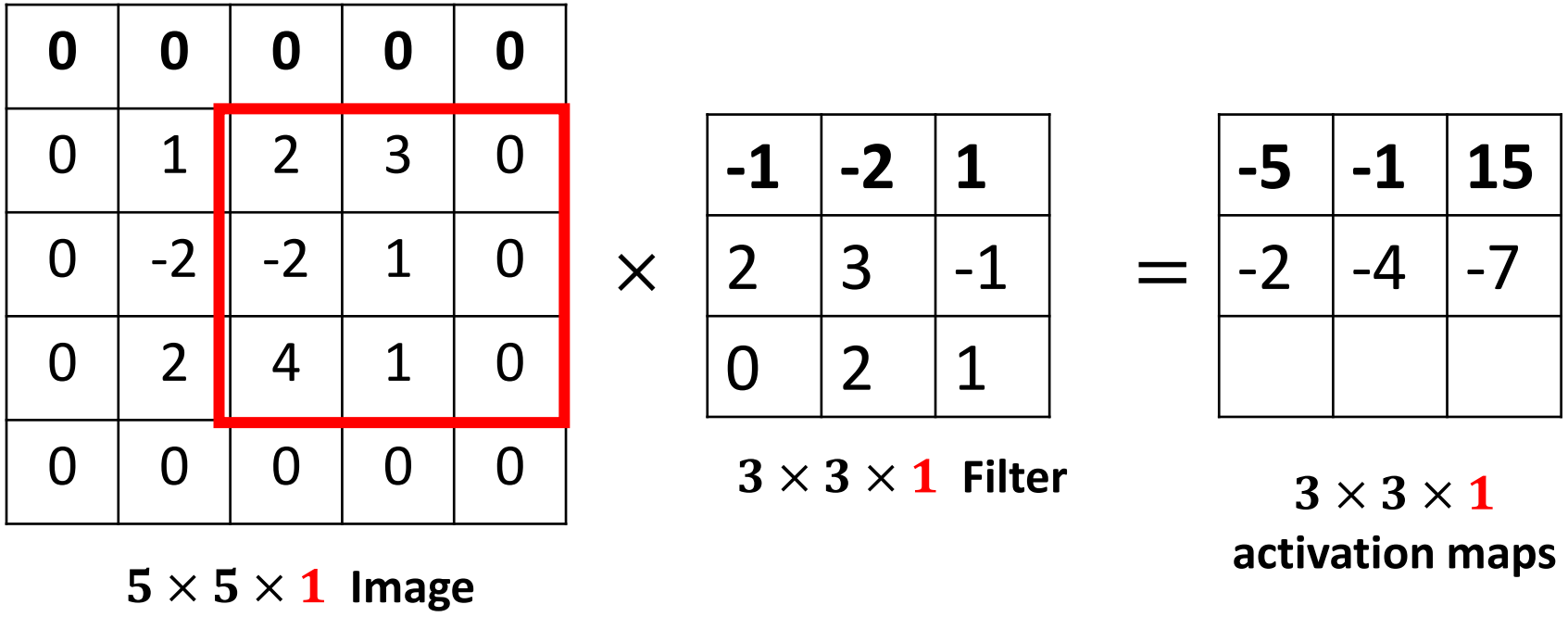
Convolutional Layer

- Example : convolution with padding (stride=1)



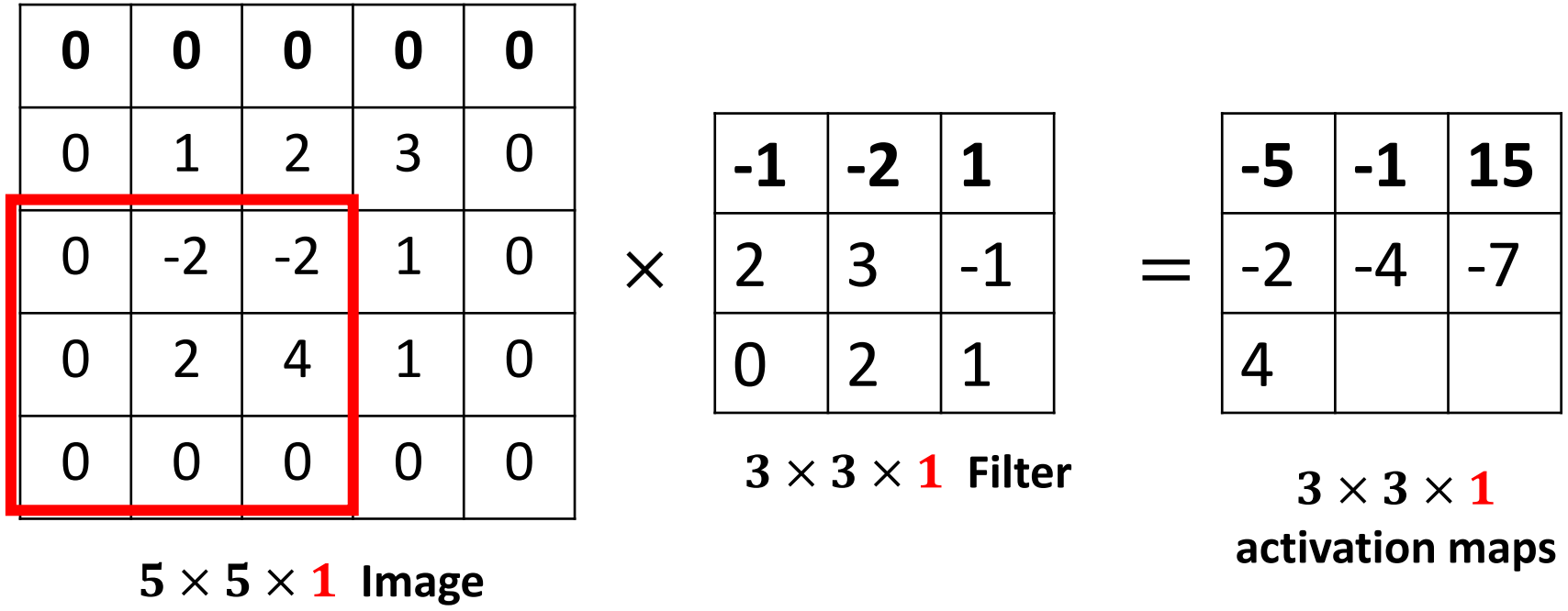
Convolutional Layer

- Example : convolution with padding (stride=1)



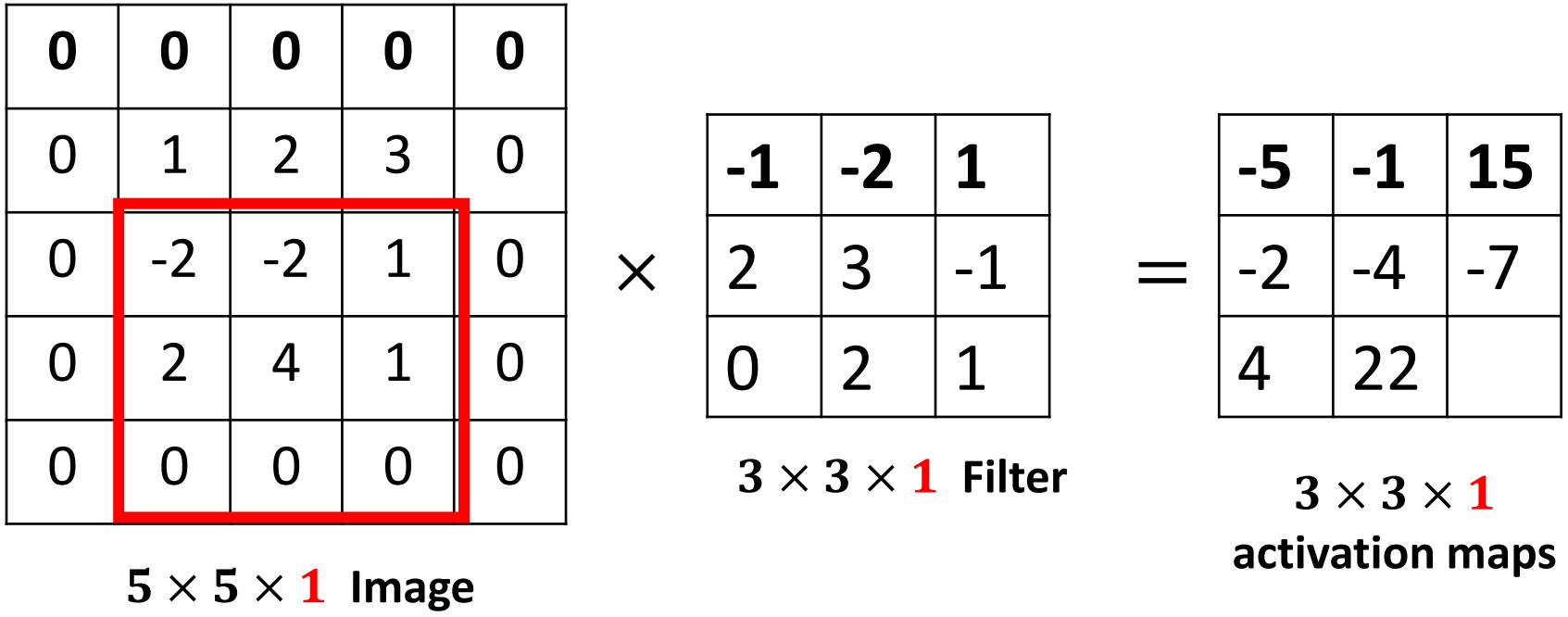
Convolutional Layer

- Example : convolution with padding (stride=1)



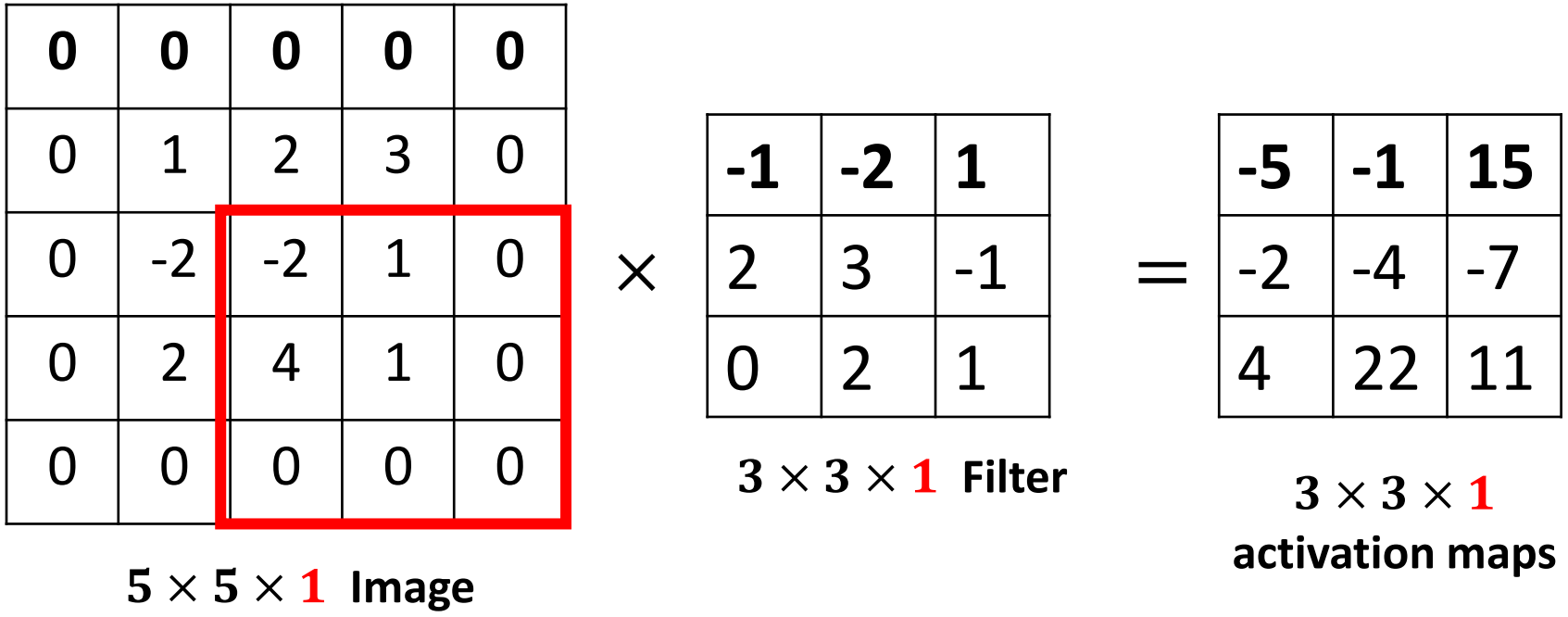
Convolutional Layer

- Example : convolution with padding (stride=1)



Convolutional Layer

- Example : convolution with padding (stride=1)



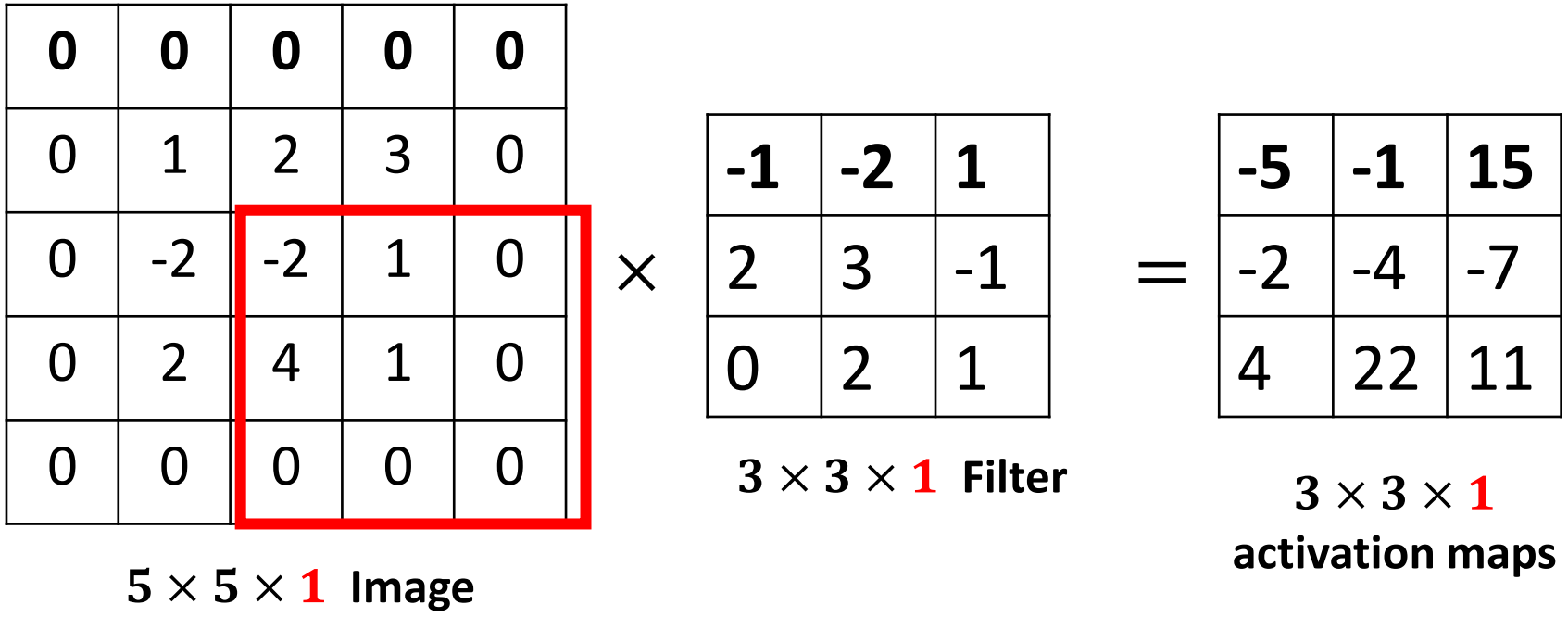
$5 \times 5 \times 1$ **Image**

$3 \times 3 \times 1$ **Filter**

$3 \times 3 \times 1$
activation maps

Convolutional Layer

- Example : convolution with padding (stride=1)



Calculate the Amount of **Necessary** Padding:

- General formula calculate the **amount of necessary padding** to ensure that concoction operation works properly:

Don't memorize these two formulas

$$P_w^{total} = \max(0, \left\lceil \frac{(W^i - 1) * S - F * W^i}{2} \right\rceil)$$

$$P_H^{total} = \max(0, \left\lceil \frac{(H^i - 1) * S - F * H^i}{2} \right\rceil)$$

P^{total}: ***Total*** padding from both sides
P:*One-side* padding

- The computed padding is applied symmetrically (i.e., half on each side). If its is odd, then it is divided unequally between the two sides. For example:
  - If calculated padding P_w is 2 , then, the padding is one on each side.
  - If calculated padding P_w is 3 (1.5 pixels on each side, rounded to 2 on one side, 1 on the other)

Calculate the Amount of desired Padding:

- General formula calculate the **amount of necessary padding** to ensure that concoction operation works properly:

Don't memorize these two formulas

$$P_w = \max(0, \frac{(W^o - 1) * S - F * W^i}{2})$$

$$P_H = \max(0, \frac{(H^o - 1) * S - F * H^i}{2})$$

- The computed padding is applied symmetrically (i.e., half on each side). If its is odd, then it is divided unequally between the two sides.
- **Example** : assume that we have $\mathbf{7 \times 5}$ image and we want convolve a $\mathbf{3 \times 3}$ filter over the image, using stride 1:
- Using the formulas above , we get $P_w = 4$ and P_H=1.
  - So, **4 pixels of total Width padding** (2 on each side for width).
  - So, **1 pixel of total Hight padding** (0.5 on each side, usually rounded to 1 on one side).

Stride with padding

- Let's assume input is $W^i \times H^i \times D$ To get the size of the output activation map $W^o \times H^o \times 1$ we use

$$W^o = \frac{(W^i - F + 2P_W)}{S} + 1$$

$$H^o = \frac{(H^i - F + 2P_H)}{S} + 1$$

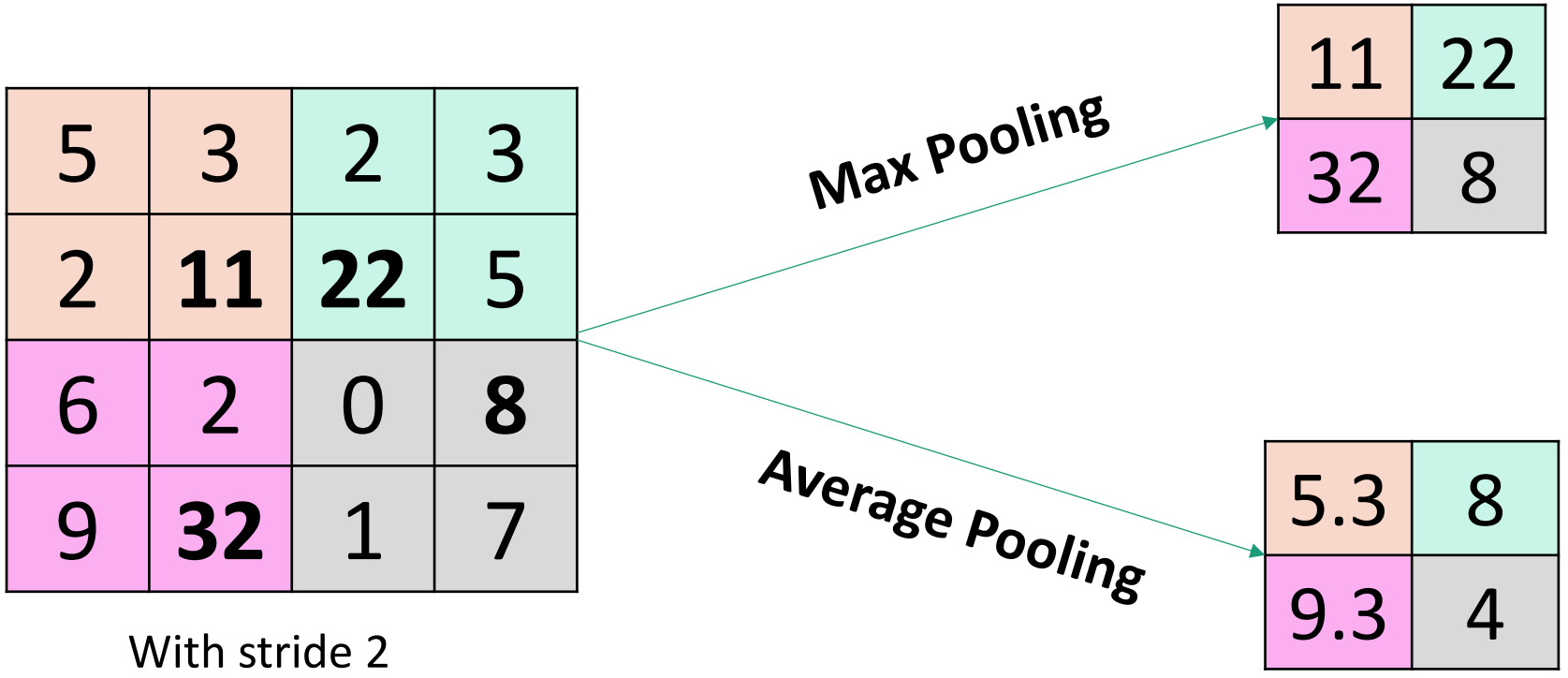
- Where F is the filter size,P_W and P_H are width and Hight zero padding (if W^iequals H^i, then P_W typically equals P_H and shortly referred to as P.
- For example,
  - $8 \times 5 \times 1$ Image, $3 \times 3 \times 1$ Filter, S =1, P =1
    - $W^o = ((8 - 3 + 2)/1) + 1 = 8$
    - $H^o = ((5 - 3 + 2)/1) + 1 = 5$

Pooling Layer

- Performs down-sampling
- Aims to:
  - **Dimensionality Reduction**: Make the size of the activation maps smaller and more manageable, and the network more efficient and faster to train..
  - **Location invariance**: Make the obtained representation less sensitive to changing the location of the features in the input.
  - **Focus on Key Features**: pooling summarizes feature maps by retaining key information (e.g., max or average values) and discarding irrelevant background details

Pooling Layer

- Two common functions used in the pooling operation: **Max Pooling and Average Pooling (pooling with stride 2)**



Max vs. Average Pooling

Max Pooling

Focus: **Strongest Feature**

It retains the **strongest activation**, highlighting the most important or dominant feature in that region (e.g., an edge, corner, or texture), helping the network focus on key patterns

Use Case:

useful in tasks where the presence of a strong feature is more critical than its precise details, such as **object detection** and **classification**.

Effect: Reduces the sensitivity to noise

Average Pooling

Focus: Overall Feature Summary

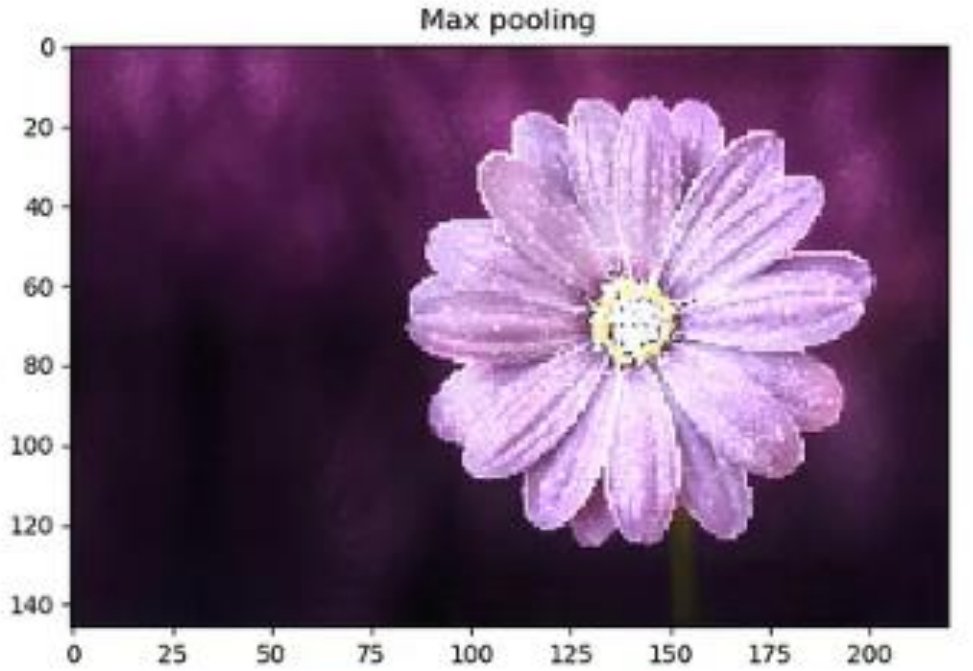
Summarizes the entire region's feature intensity, providing a smoother representation by considering all activations equally.

Use Case:

- Used in tasks that need a more complete view of the feature map, such as image reconstruction (e.g., improve image resolution), ensures that subtle features (e.g., textures or small objects) are not lost, and also retain smoother transition and avoid feature over-sharpening.
- Commonly used in transition layers or after final feature extraction to reduce feature map size while preserving information.

Effect: Keeps more contextual and background info.

Max vs. Average Pooling



Max pooling vs applying ReLU

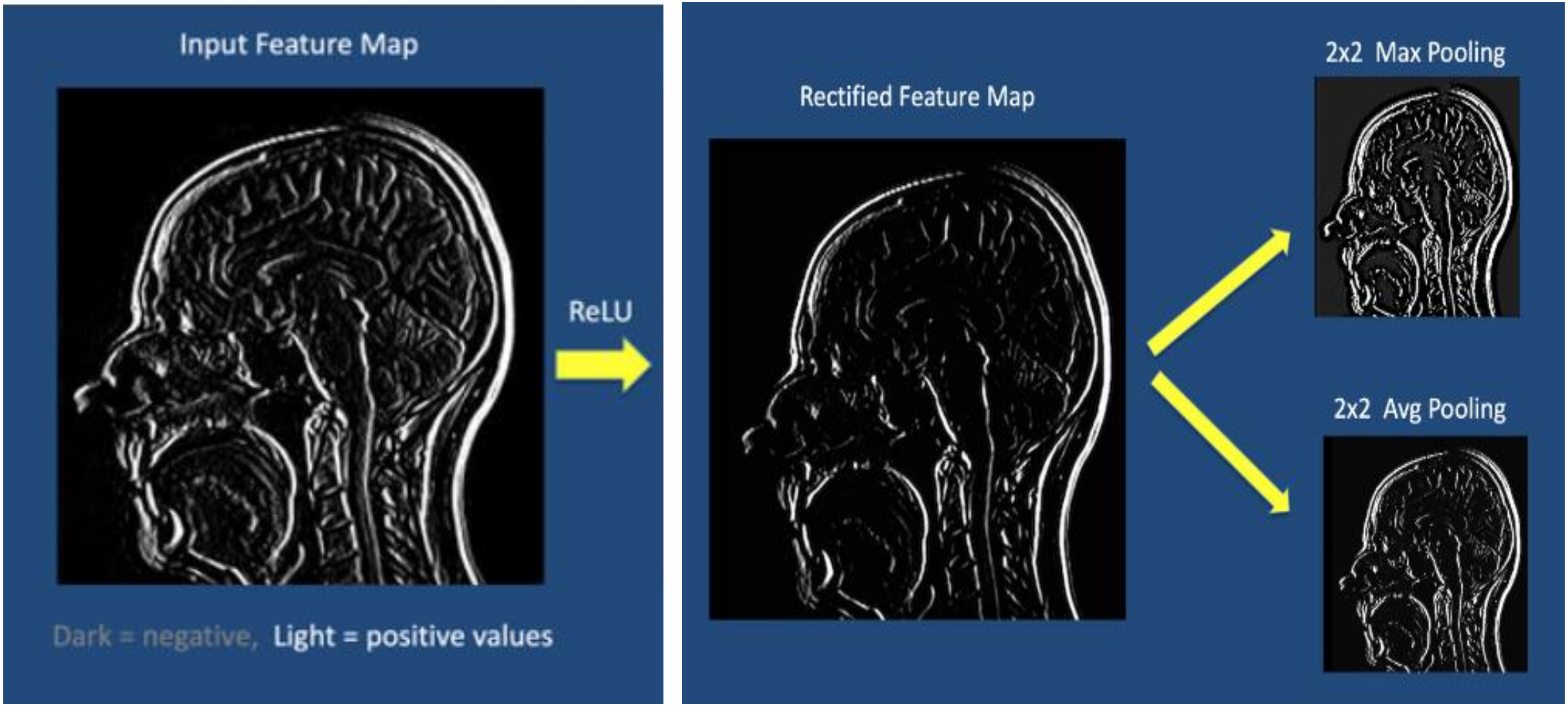
Key Difference:

- **Max Pooling**: Operates at the spatial level by summarizing feature values within a local region.
- **ReLU**: Operates element-wise, modifying individual pixel/activation values independently.

Similarities

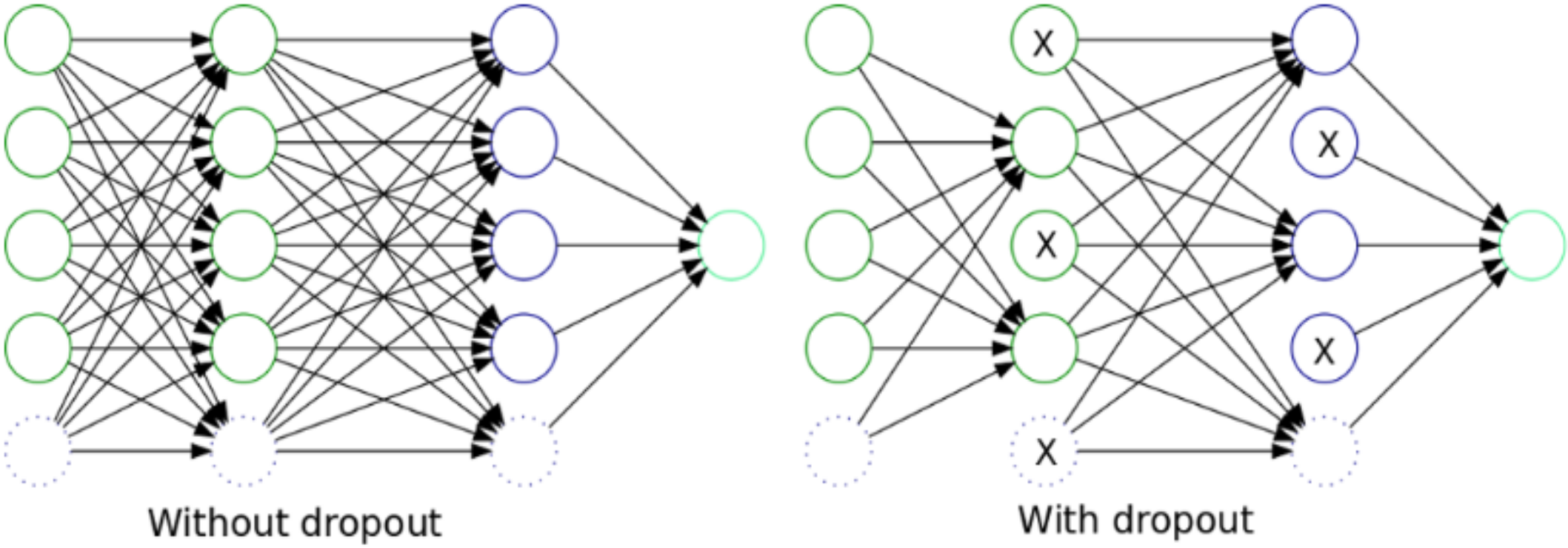
- Focus on Positive or Strong Activations.
- **Introduce Sparsity:** *Max Pooling* results in a **sparser feature map** by keeping only one value (**the maximum**) per pooling region, while *ReLU* creates sparsity by zeroing out all **negative activations.**
- **Reduce Noise: Max Pooling** ignores smaller values in the pooling region, while **ReLU** cancels negative activations.

Usually Combined!


Input Feature Map
ReLU
Dark = negative, Light = positive values
Rectified Feature Map
2x2 Max Pooling
2x2 Avg Pooling

Dropout in Neural Networks

Dropout is a regularization technique used in neural networks to prevent *overfitting* by randomly "***dropping out***" (setting to zero) a proportion of neurons during each training iteration.



Dropout in Neural Networks

How it works?

- During training, a specified percentage (e.g., 20% or 50%) of neurons in a layer are randomly ignored (i.e., their output is set to zero).

Purpose: forces the network to rely on different combinations of neurons during each iteration, preventing the network from becoming overly dependent on specific neurons.

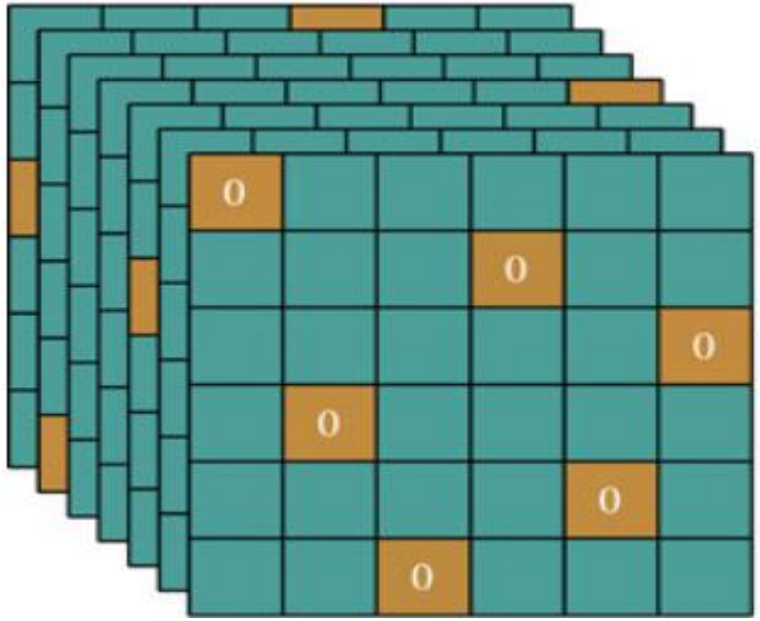
Dropout during training vs. testing:

- **Training**: Dropout is applied, and neurons are randomly dropped.
- **Testing**: Dropout is not applied, but the neuron outputs are scaled (by the dropout rate) to account for the randomness during training.

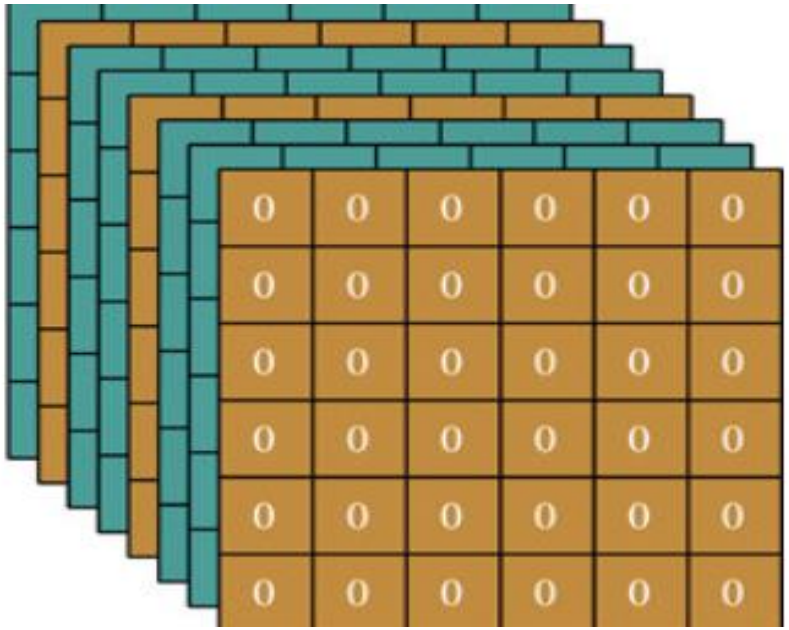
Usage: Applied to fully connected layers, and sometimes in convolutional layers.
dropout rates is a hyper-parameter, typically ranges from 0.2 to 0.5.

Applying Dropout to Convolutional layers

Two common methods:



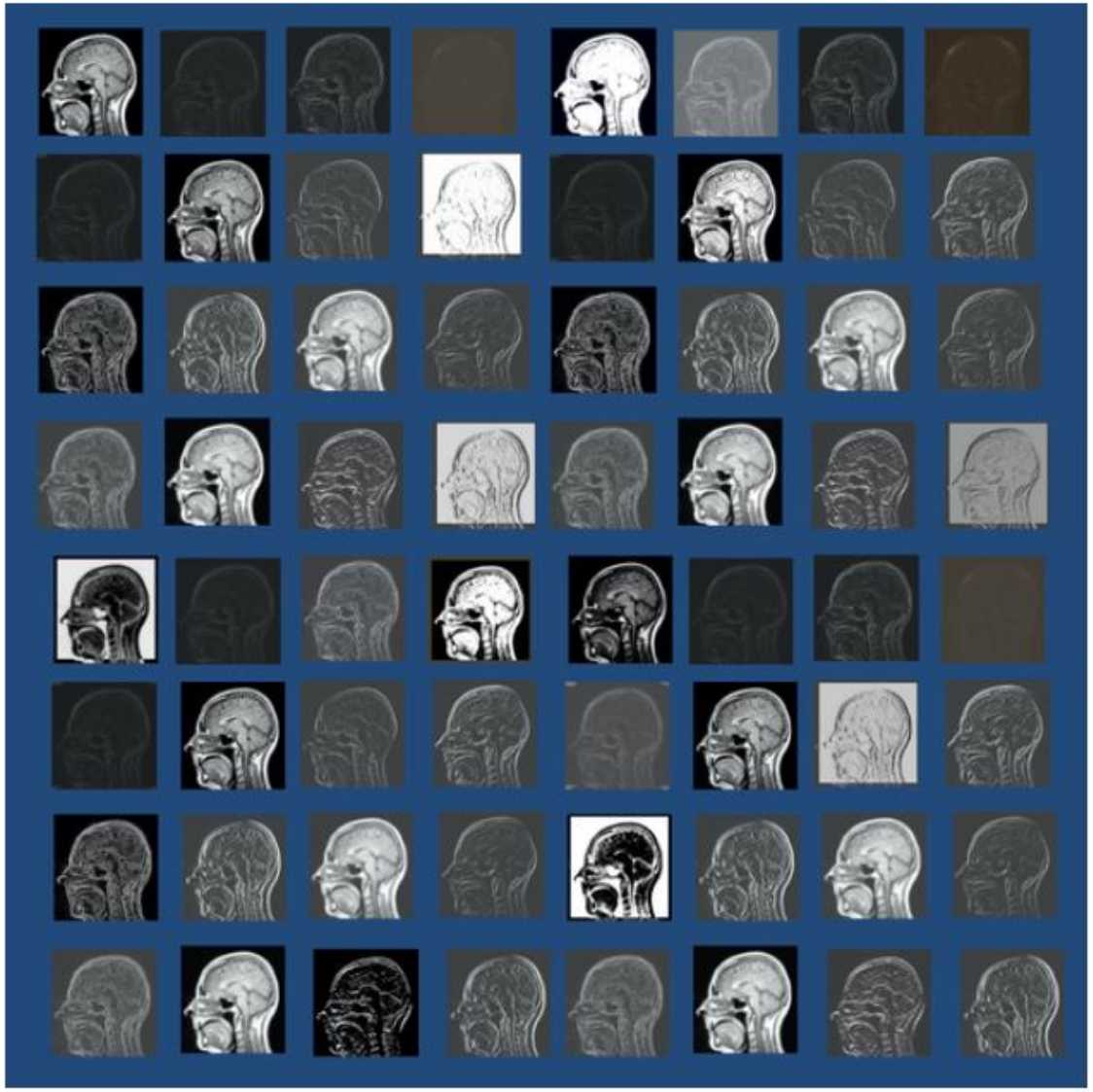
Standard Dropout: similar to fully connected layers, dropout (set to zero) is applied element-wise **on Individual Activations** according to a certain probability.



Spatial Dropout (Recommended): Instead of dropping individual activations, entire feature maps according to a certain probability (20% of the feature maps are entirely set to zeros).

Why many filters?

To detect a wide variety of features!

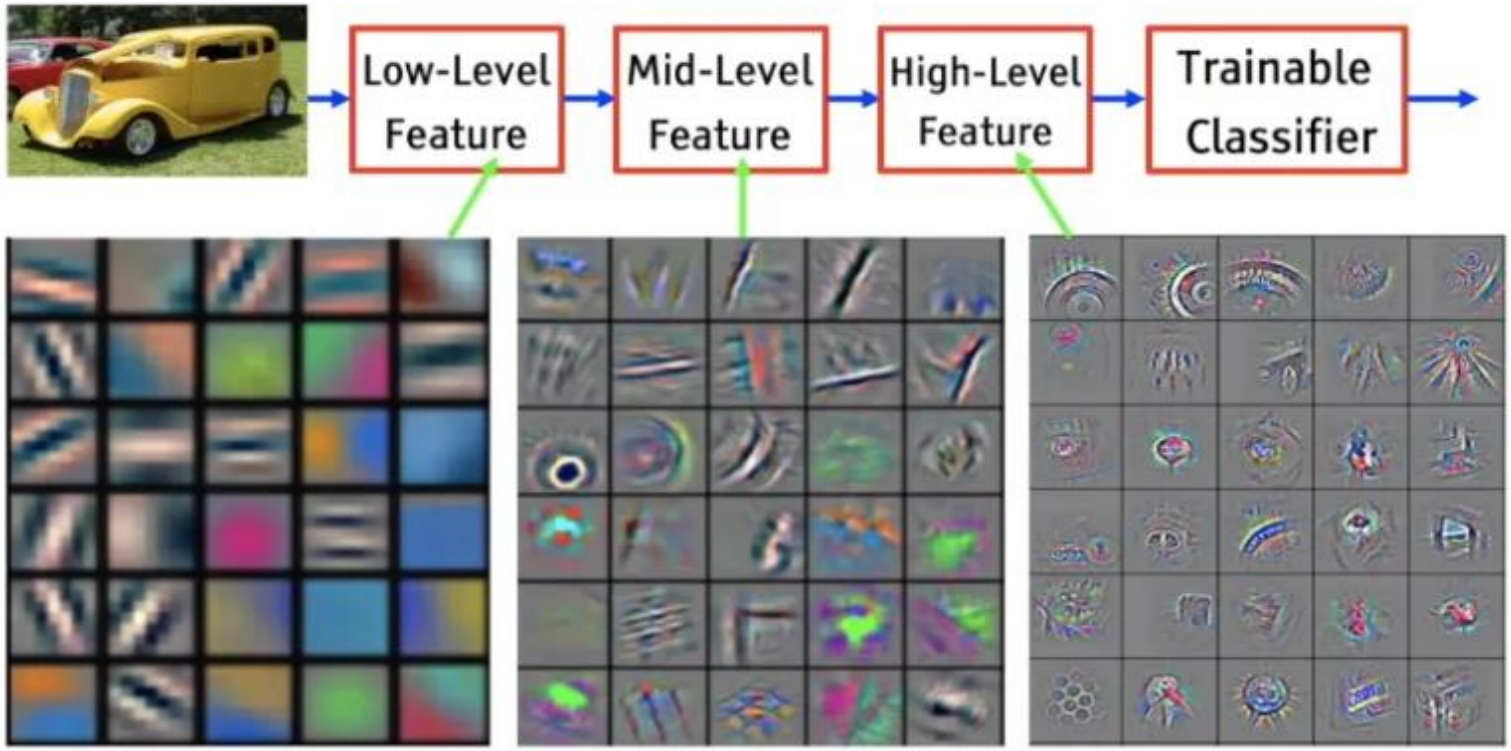


First-level feature maps of a brain MRI extracted from a convolutional network. Each small image is the result of convolution with one of 64 different first-level filters that emphasize various simple properties, including bright vs dark regions, edges, curves, and shadows.

Why many Layers?

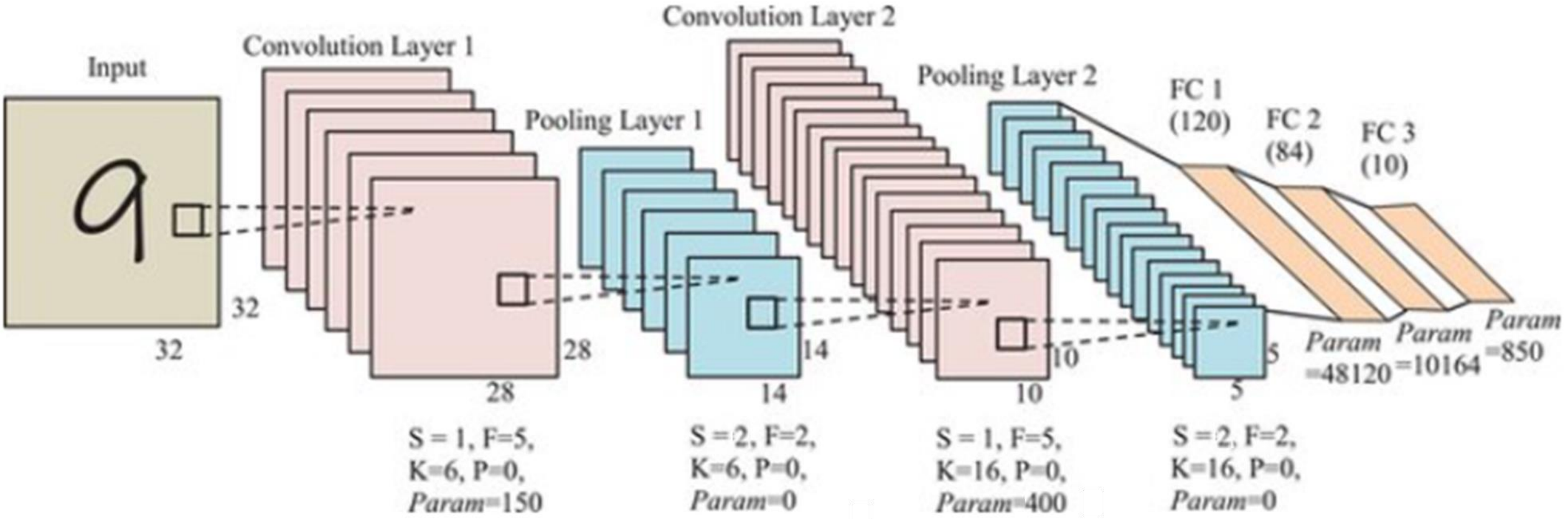
To allow for a hierarchical feature extraction!

- **Low-Level Features** in Early Layers: edges, corners, and gradients.
- **Mid-Level Features** in Intermediate Layers: shapes, textures, and object parts (like eyes or wheels).
- **High-Level Features** in Deeper Layers: complex patterns or semantic features, such as identifying entire objects (e.g., a car, a face).



An example of how different filters in convolutional neural networks look to obtain visual features or image components.

A CNN for Digit Recognition



PyTorch code to build and train the CNN described in the previous slide

PyTorch code to build and train the CNN described in the previous slide

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
```

```python
# Define the Custom CNN model based on the provided architecture
class CustomCNN(nn.Module):
    def __init__(self):
        super(CustomCNN, self).__init__()
        # First Convolutional Layer: Input 1 channel, K=6, F=5, S=1, P=0
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1, padding=0)
        # First Pooling Layer: K=6, Pooling F=2, S=2
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        # Second Convolutional Layer: Input 6 channels, K=16, F=5, S=1, P=0
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0)
        # Second Pooling Layer: K=16, Pooling F=2, S=2
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        # Fully Connected Layers
        self.fc1 = nn.Linear(16 * 5 * 5, 120)  # First Fully Connected Layer
        self.fc2 = nn.Linear(120, 84)          # Second Fully Connected Layer
        self.fc3 = nn.Linear(84, 10)           # Output Layer for 10 classes

    def forward(self, x):
        x = F.relu(self.conv1(x))  # Convolution Layer 1 + ReLU
        x = self.pool1(x)          # Pooling Layer 1
        x = F.relu(self.conv2(x))  # Convolution Layer 2 + ReLU
        x = self.pool2(x)          # Pooling Layer 2
        x = x.view(-1, 16 * 5 * 5) # Flatten the feature maps for the fully connected layer
        x = F.relu(self.fc1(x))    # Fully Connected Layer 1 + ReLU
        x = F.relu(self.fc2(x))    # Fully Connected Layer 2 + ReLU
        x = self.fc3(x)            # Output Layer
        return x
```

```python
# Define transformations and load MNIST dataset
transform = transforms.Compose([
    transforms.Pad(padding=2),  # Add 2 pixels of padding to make 28x28 -> 32x32
    transforms.ToTensor()
])

# Load full training dataset
full_train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

# Split the training dataset into 90% training and 10% validation
train_size = int(0.9 * len(full_train_dataset))
val_size = len(full_train_dataset) - train_size
train_dataset, val_dataset = random_split(full_train_dataset, [train_size, val_size])

# Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# Initialize the model, loss function, and optimizer
model = CustomCNN()
criterion = nn.CrossEntropyLoss()  # CrossEntropyLoss for classification
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```python
# Training function
def train_model(model, train_loader, val_loader, criterion, optimizer, epochs=5):
    for epoch in range(epochs):
        # Training Phase
        model.train()
        running_loss = 0.0
        for images, labels in train_loader:
            optimizer.zero_grad()  # Zero gradients
            outputs = model(images)  # Forward pass
            loss = criterion(outputs, labels)  # Compute loss
            loss.backward()  # Backward pass
            optimizer.step()  # Update parameters
            running_loss += loss.item()

        # Validation Phase
        model.eval()
        val_loss = 0.0
        correct = 0
        total = 0
        with torch.no_grad():
            for images, labels in val_loader:
                outputs = model(images)
                loss = criterion(outputs, labels)
                val_loss += loss.item()
                _, predicted = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        val_accuracy = 100 * correct / total
        print(f"Epoch [{epoch+1}/{epochs}], Training Loss: {running_loss/len(train_loader):.4f}, "
              f"Validation Loss: {val_loss/len(val_loader):.4f}, Validation Accuracy: {val_accuracy:.2f}%")
```

```python
# Testing function
def test_model(model, test_loader):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():  # Disable gradient computation for testing
        for images, labels in test_loader:
            outputs = model(images)  # Forward pass
            _, predicted = torch.max(outputs, 1)  # Get predictions
            total += labels.size(0)  # Total samples
            correct += (predicted == labels).sum().item()  # Correct predictions
    print(f"Test Accuracy: {100 * correct / total:.2f}%")

# Train and validate the model
train_model(model, train_loader, val_loader, criterion, optimizer, epochs=5)

# Test the model
test_model(model, test_loader)
```

Adding dropout

- The code in the previous slides doesn't include using dropout!
- You can add it as follows:

```
    # First Convolutional Layer: Input 1 channel, K=6, F=5, S=1, P=0
    self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1, padding=0)
    self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)  # First Pooling Layer
    # Second Convolutional Layer: Input 6 channels, K=16, F=5, S=1, P=0
    self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0)
    self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)  # Second Pooling Layer
            # Fully Connected Layers with Dropout
    self.fc1 = nn.Linear(16 * 5 * 5, 120)  # First Fully Connected Layer
    self.dropout1 = nn.Dropout(p=0.5)      # Dropout after first FC layer
    self.fc2 = nn.Linear(120, 84)          # Second Fully Connected Layer
    self.dropout2 = nn.Dropout(p=0.5)      # Dropout after second FC layer
    self.fc3 = nn.Linear(84, 10)           # Output Layer for 10 classes
def forward(self, x):
    x = F.relu(self.conv1(x))  # Convolution Layer 1 + ReLU
    x = self.pool1(x)          # Pooling Layer 1
    x = F.relu(self.conv2(x))  # Convolution Layer 2 + ReLU
    x = self.pool2(x)          # Pooling Layer 2
    x = x.view(-1, 16 * 5 * 5)
    x = F.relu(self.fc1(x))    # Fully Connected Layer 1 + ReLU
    x = self.dropout1(x)       # Apply Dropout
    x = F.relu(self.fc2(x))    # Fully Connected Layer 2 + ReLU
    x = self.dropout2(x)       # Apply Dropout
    x = self.fc3(x)            # Output Layer
    return x
```

Training the CNN for 5 epochs

Without dropout:

```
Epoch [1/5], Training Loss: 0.3004, Validation Loss: 0.0963, Validation Accuracy: 97.03%
Epoch [2/5], Training Loss: 0.0842, Validation Loss: 0.0687, Validation Accuracy: 97.87%
Epoch [3/5], Training Loss: 0.0602, Validation Loss: 0.0546, Validation Accuracy: 98.13%
Epoch [4/5], Training Loss: 0.0449, Validation Loss: 0.0532, Validation Accuracy: 98.25%
Epoch [5/5], Training Loss: 0.0365, Validation Loss: 0.0425, Validation Accuracy: 98.58%
Test Accuracy: 98.88%
```

With dropout:

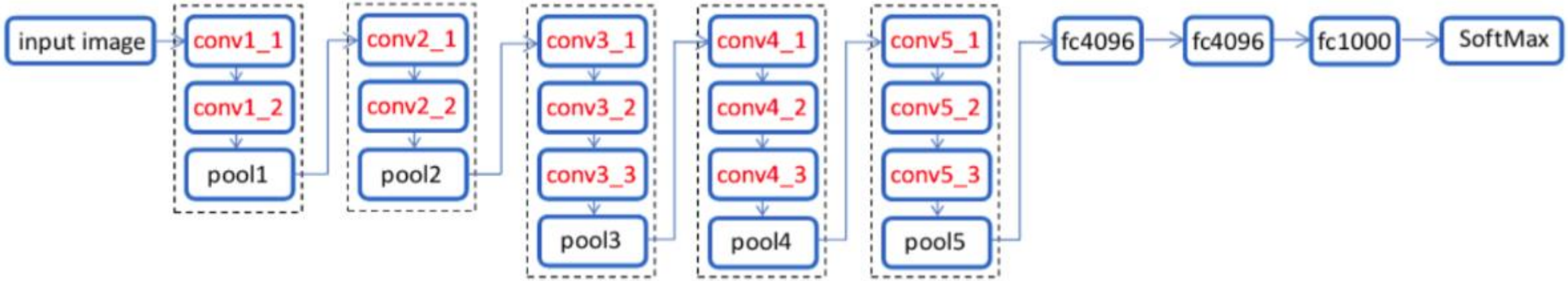
```
Epoch [1/5], Training Loss: 0.5258, Validation Loss: 0.1049, Validation Accuracy: 97.00%
Epoch [2/5], Training Loss: 0.1674, Validation Loss: 0.0722, Validation Accuracy: 98.02%
Epoch [3/5], Training Loss: 0.1285, Validation Loss: 0.0724, Validation Accuracy: 98.00%
Epoch [4/5], Training Loss: 0.1062, Validation Loss: 0.0630, Validation Accuracy: 98.30%
Epoch [5/5], Training Loss: 0.0926, Validation Loss: 0.0504, Validation Accuracy: 98.67%
Test Accuracy: 98.79%
```

A reduced dropout rate (e.g., 0.3) should be applied, or more epochs may be needed.
Alternatively, dropout might be harmful and unnecessary for this task. More hyper-parameter tuning is needed!

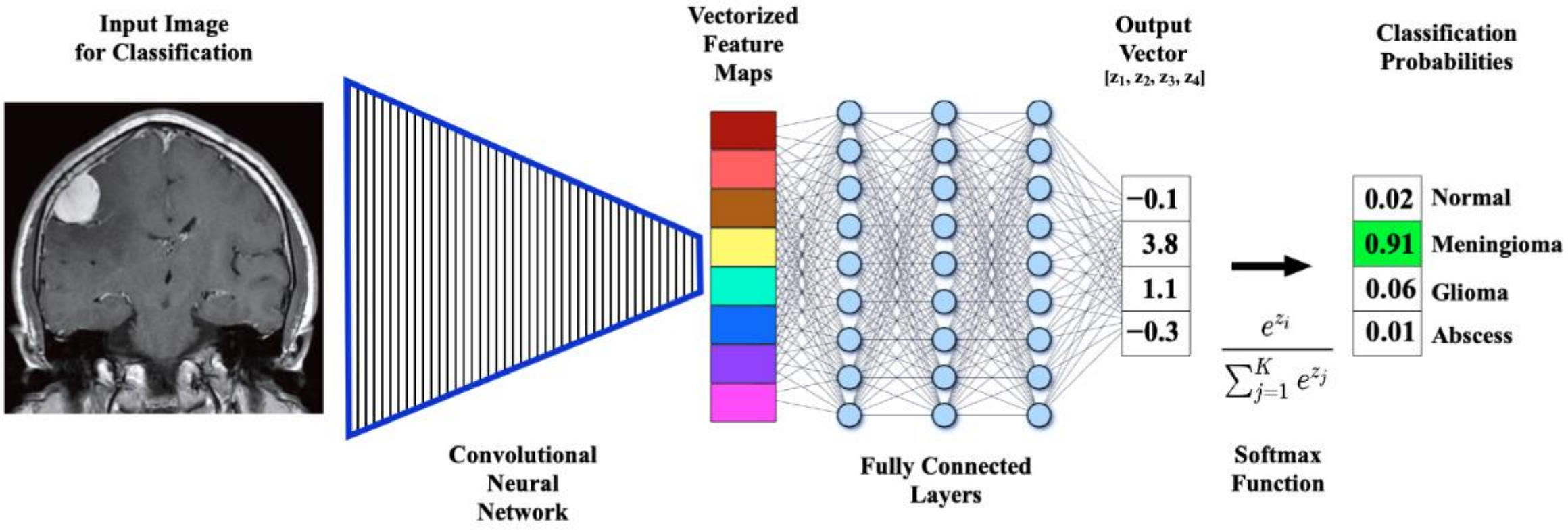
Alex vs VGG for Image classification

Alex (Sutskever et al. 2012)

VGG (Simonyan et al. 2014)



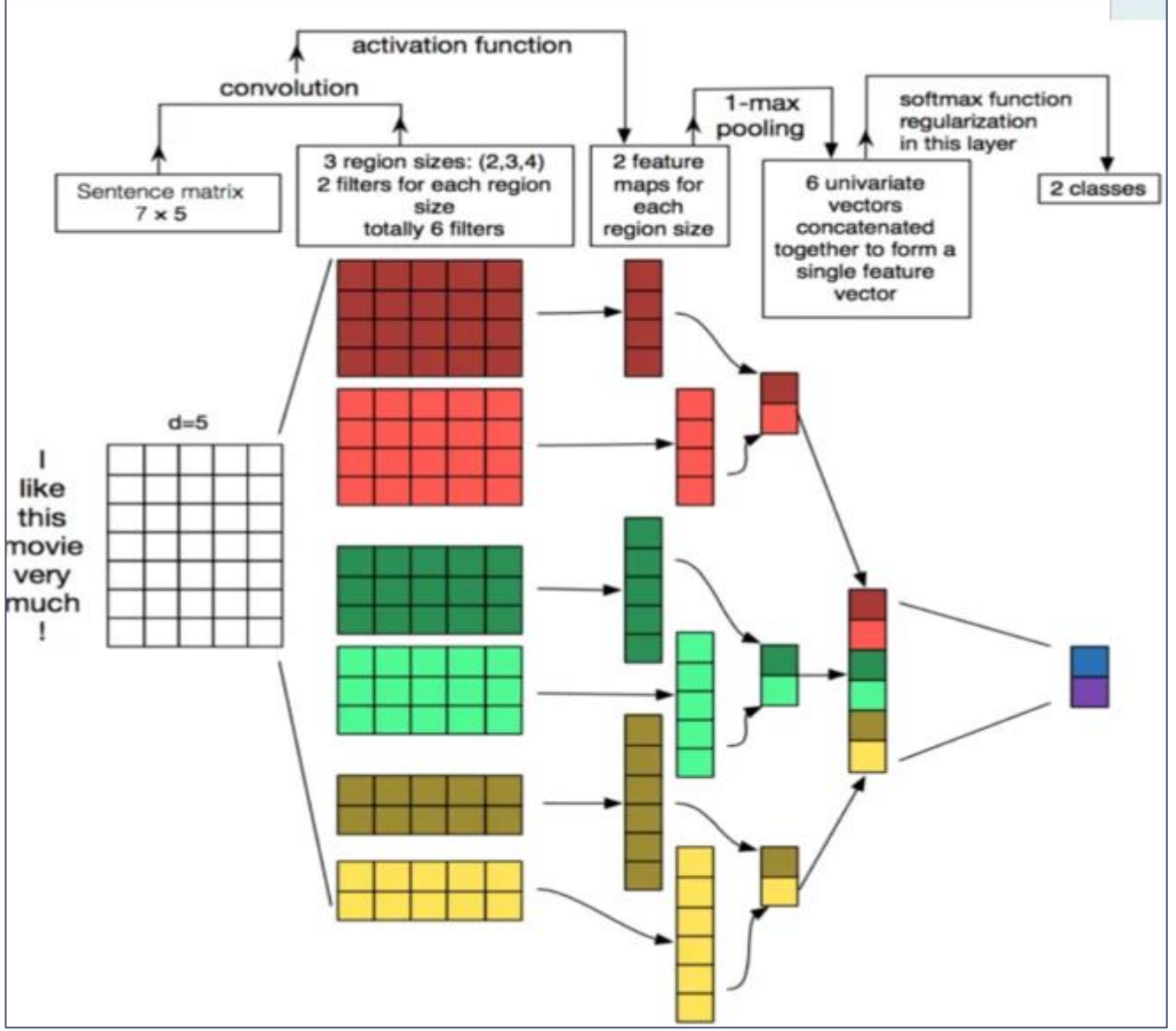
CNN for Medical Image Classification



CNN for Sentence Classification

The input is a sentence of m words (0 padding probably applied) each word is embedded into a fixed-dimensional vector (e.g., $d = 5$) forming a $m \times 5$ matrix (e.g., $m = 7$).

1. **Convolution Operation**: Multiple filters with different region sizes (2×5, 3×5 and 4×5) slide over the sentence matrix. Each filter extracts specific n-gram features, resulting in feature maps. Here, 2 filters per region size are used, generating a total of 6 feature maps.
2. **Activation and Pooling**:
   After applying a non-linear activation function (e.g., ReLU), 1D-max pooling is performed over each feature map. This operation selects the most important feature from each map, compressing them into univariate vectors.
3. **Feature Vector and Classification**:
   The 6 univariate vectors from pooling are concatenated into a single feature vector. This vector is then passed to a fully connected layer with a **Softmax (or sigmoid)** function for final classification, predicting one of two possible classes.



References

- LeCun, Yann, et al. "Gradient-based learning applied to document recognition." *Proceedings of the IEEE* 86.11 (1998): 2278-2324.
- Ciresan, Dan Claudiu, et al. "Flexible, high performance convolutional neural networks for image classification." *Twenty-Second International Joint Conference on Artificial Intelligence*. 2011.
- Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems*. 2012.
- https://dataconomy.com/2017/04/history-neural-networks/

Questions?